



DIGITAL DESIGN & COMPUTER ORGANISATION

Multiplication-1.1

Team DDCO

Department of Computer Science &
Engineering

DIGITAL DESIGN & COMPUTER ORGANISATION

Multiplication - 1

Team DDCO

Department of Computer Science &
Engineering

MULTIPLICATION - 1

Course Outline



- ❖ Digital Design
 - Combinational logic design
 - Sequential logic design
 - Multiplication - 1
- ❖ Computer Organisation
 - Architecture (microprocessor instruction set)
 - Microarchitecture (microprocessor operation)

Concepts covered

- ❖ Binary Multiplication

MULTIPLICATION - 1

Array Multiplication- T2-section 6.3

- ❖ Manual Method of multiplication of 2 decimal numbers
- ❖ Similarly we can accomplish multiplication of 2 binary numbers



MULTIPLICATION - 1

Array Multiplication

- ❖ Manual Method of multiplication of 2 decimal numbers
- ❖ Similarly we can accomplish multiplication of 2 binary numbers

$ \begin{array}{r} 13 \\ \times 11 \\ \hline 13 \\ 13 \\ \hline 143 \end{array} $	$ \begin{array}{r} 1101 \\ \times 1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 10001111 \end{array} $	(13) Multiplicand, M (11) Multiplier, Q (143) Product, P
---	--	--

- This algorithm applies to unsigned numbers or positive signed numbers.
- The product of two n -digit numbers will result in $2n$ digit result.

- Multiplication of binary numbers can be represented using AND function.
- The product can be obtained by adding partial product in each stage.
- Multiplication of binary numbers can be achieved using an array of combinational elements.

MULTIPLICATION

- 1

Array

Multiplication

1 1 0 1	(13) Multiplicand, M
X 1 0 1 1	(11) Multiplier, Q

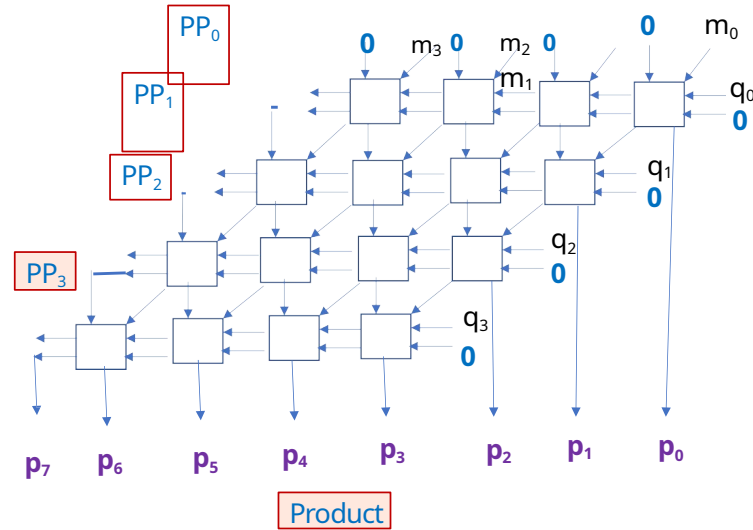
0 0 0 0	Initial Partial Product, PP ₀
1 1 0 1	

1 1 0 1	Partial Product, PP ₁
1 1 0 1	

1 0 0 1 1 1	Partial Product, PP ₂

1 0 0 1 1 1	Partial Product, PP ₃
1 1 0 1	

1 0 0 0 1 1	(143) Product, P
1 1	

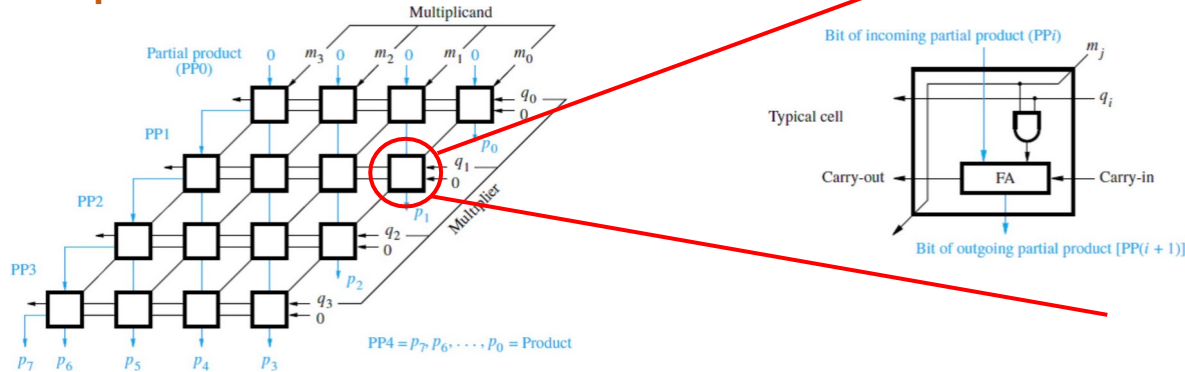


MULTIPLICATION

- 1

Array

Multiplication

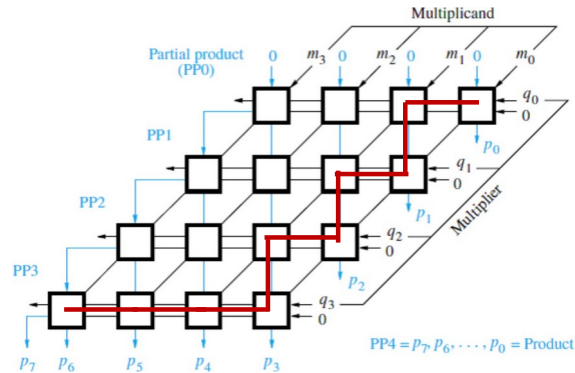


- AND gate in each cell performs $m_j \cdot q_i$ product
- A Full Adder adds this with the previous partial product bit PP_i along with previous bit Carry-in and generates $PP_{(i+1)}$ and Carry-out.
- m_j is shifted to downward cell whereas q_i is connected to all the cells in a row.

MULTIPLICATION

- 1

Think about it



- What is the worst case propagation delay?
- Assume FA has 2 gate delays and all AND gates has 1 gate delay. Also, the first row can have only AND gates.
- How many gates are required for $m \times n$ array multiplication?

Each of the 2 FA blocks in row 2 to n-1 introduces 2 gate delays, for a total of $4(n-2)$ gate delays. Row n introduces $2n$ gate delays. Adding the initial and gate delay for row 1 and all other cells, total delay is:

$$4(n-2)+2n+1=6n-8+1=6(n-1)-1$$

Array Multiplication

								Inputs
				A3	A2	A1	A0	
				×	B3	B2	B1	B0
								Internal Signals
				C	$B0 \times A3$	$B0 \times A2$	$B0 \times A1$	$B0 \times A0$
				+	$B1 \times A3$	$B1 \times A2$	$B1 \times A1$	$B1 \times A0$
				C	Sum	Sum	Sum	Sum
				+	$B2 \times A3$	$B2 \times A2$	$B2 \times A1$	$B2 \times A0$
				C	Sum	Sum	Sum	Sum
				+	$B3 \times A3$	$B3 \times A2$	$B3 \times A1$	$B3 \times A0$
				C	Sum	Sum	Sum	Sum
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	Outputs

MCQ

In a 4×4 array multiplier, how many Full Adders (FA) and AND gates are required?

- A. 16 AND + 12 FA
- B. 16 AND + 8 FA
- C. 12 AND + 16 FA
- D. 8 AND + 12 FA

For an $n \times n$ array multiplier using **AND** and **Full Adder** cells, each FA has 2 gate delays and each AND gate has 1 gate delay.

What is the **worst-case propagation delay** (in gate delays)?

- A. $2n + 1$
- B. $4(n - 2) + 2n + 1$
- C. $6(n - 1) - 1$
- D. $n^2 + 1$

MCQ

In a 4×4 array multiplier, how many Full Adders (FA) and AND gates are required?

- A. 16 AND + 12 FA
- B. 16 AND + 8 FA
- C. 12 AND + 16 FA
- D. 8 AND + 12 FA

Answer: A $\rightarrow$ ($n^2 = 16$ AND gates , $FA = n \times (n - 1) = 12$)

For an $n \times n$ array multiplier using **AND** and **Full Adder** cells, each FA has 2 gate delays and each AND gate has 1 gate delay.

What is the **worst-case propagation delay** (in gate delays)?

- A. $2n + 1$
- B. $4(n - 2) + 2n + 1$
- C. $6(n - 1) - 1$
- D. $n^2 + 1$

Answer: C $\rightarrow$ Delay = $6(n - 1) - 1$

MCQ

For two unsigned n -bit numbers multiplied using array multiplication, the result has —

- A. n bits
- B. $n + 1$ bits
- C. $2n$ bits
- D. n^2 bits

In a 4×4 array multiplier, if one AND gate delay = 1 ns and one FA delay = 2 ns, estimate the worst-case delay.

- A. 20ns
- B. 18 ns
- C. 17 ns
- D. 25 ns

MCQ

For two unsigned n -bit numbers multiplied using array multiplication, the result has —

- A. n bits
- B. $n + 1$ bits
- C. $2n$ bits
- D. n^2 bits

Answer: C → Product of two n -bit numbers = $2n$ -bit result

In a 4×4 array multiplier, if one AND gate delay = 1 ns and one FA delay = 2 ns, estimate the worst-case delay.

- A. 20ns
- B. 18 ns
- C. 17 ns
- D. 25 ns

Answer: C → Delay = $6(n - 1) - 1 = 17$ gate delays ≈ 17 ns



THANK YOU

Team DDCO

Department of Computer Science &
Engineering



DIGITAL DESIGN & COMPUTER ORGANISATION

Multiplication-2

Team DDCO

Department of Computer Science &
Engineering

DIGITAL DESIGN & COMPUTER ORGANISATION

Multiplication

Team DDCO.

Department of Computer Science & Engineering

MULTIPLICATION-2

Course Outline

- ❖ Digital Design
 - Combinational logic design
 - Sequential logic design
 - Multiplication - 1
- ❖ Computer Organisation
 - Architecture (microprocessor instruction set)
 - Microarchitecture (microprocessor operation)

Concepts covered

- Shift-Add
- Multiplication Booth Algorithm

MULTIPLICATION-2

Shift - Add Multiplication T2- section 6.3

❖ **Array Multiplier**

- > Very hardware intensive
- > Expensive & waste of resources

❖ **Multicycle (Shift-Add) Multiplier**

- > Add & Shift the multiplicand
- > Iterate the process through the bits of multiplier

```

      1 1 0 1      (13) Mul@plicand, M
    X 1 0 1 1      (11) Mul@plier, Q
    -----
      1 1 0 1
      1 1 0 1
      0 0 0 0
    --- 1 1 0 1 ---
    1 0 0 0 1 1 1 1      (143) Product, P
  
```

MULTIPLICATION-2

Shift - Add Multiplication

Sequential multiplication Algorithm

1. Initialize $2n$ -bit Registers for two n -bit unsigned multiplication register M with multiplicand, register Q with multiplier, register A with Zero (0) and Carry flag with Zero(0).
2. If LSB of register Q i.e., $q_0 = 1$, then Add $A=A+M$ and retain Q otherwise if $q_0 = 0$ then No-Add.
3. Right Shift 1 bit from Carry Flag(C) to Register Q .
4. Repeat Step 2 and 3 n times

MULTIPLICATION-2

Shift - Add Multiplication

❖ Array Multiplier

- Very hardware intensive
- Expensive & waste of resources

❖ Multicycle (Shift-Add) Multiplier

- Add & Shift the multiplicand
- Iterate the process through the bits of multiplier

$$\begin{array}{r}
 1101 \quad (13) \text{ Mul@plicand, M} \\
 \times 1011 \quad (11) \text{ Mul@plier, Q} \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 1101 \\
 \hline
 10001111 \quad (143) \text{ Product, P}
 \end{array}$$

1101	
X 1011	
0000	Initial result
1101	Multiplier = 1; Add Multiplicand
<u>1101</u>	Shift Right
1101	Multiplier = 1; Add Multiplicand
1101	
<u>100111</u>	Shi< Right
100111	Mul@plier = 0; No Add
<u>100111</u>	Shift Right
100111	Mul@plier = 1; Add Mul@plicand
1101	
10001111	(143) Product, P

MULTIPLICATION-2

Shift - Add Multiplication

Implementation

1. Use an Adder circuitry to do repeated addition.
2. Multiplication through spatial addition, requires a single n-bit adder doing addition n times.
3. After every addition, the result has to be shifted right once.
4. Monitor the multiplier bit to add multiplicand to the partial product. Repeat the add – shift right process for the number of multiplier bits.
5. All the control signals has to be generated for this sequential process appropriately, by a control sequencer.
6. Requires n-cycles for n times addition.
7. This technique is used for unsigned binary numbers multiplication or positive signed binary numbers multiplication.

MULTIPLICATION-2

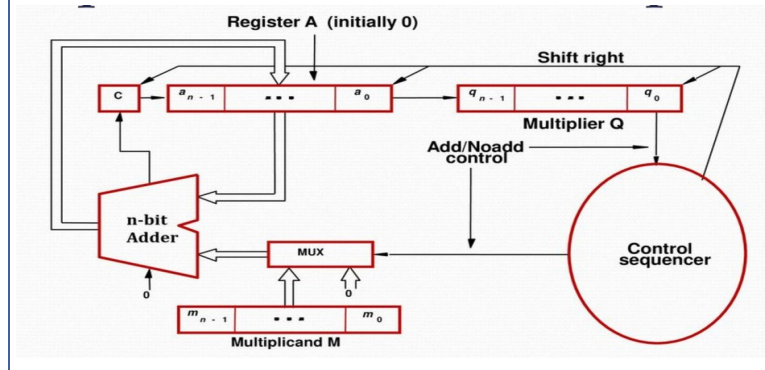
Shift - Add Multiplication

- ❖ We Require:
 - Accumulator register – A
 - Multiplier – Q
 - Multiplicand – M
 - N-bit Adder
 - Control signals for Shift and Add

MULTIPLICATION-2

Shift - Add Multiplication

- ❖ We Require:
 - Accumulator register – A
 - Multiplier – Q
 - Multiplicand – M
 - N-bit Adder
 - Control signals for Shift and Add

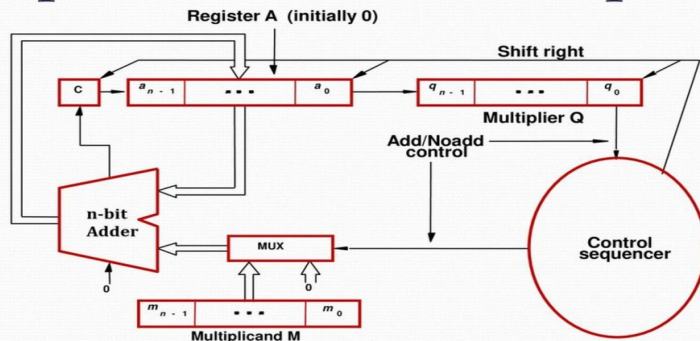


MULTIPLICATION-2

Shift - Add Multiplication

❖ We Require:

- Accumulator register – A
- Multiplier – Q
- Multiplicand – M
- N-bit Adder
- Control signals for Shift and Add



	M		
	1 1 0 1		
0	0 0 0 0	1 0 1 1	
C	A	Q	
0	1 1 0 1	1 0 1 1	$Q_0=1$, Add M
0	0 1 1 0	1 1 0 1	Shift Right
1	0 0 1 1	1 1 0 1	$Q_0=1$, Add M
0	1 0 0 1	1 1 1 0	Shift Right
0	1 0 0 1	1 1 1 0	$Q_0=0$, No Add
0	0 1 0 0	1 1 1 1	Shi< Right
1	0 0 0 1	1 1 1 1	$Q_0=1$, Add M
0	1 0 0 0	1 1 1 1	Shi< Right
	(143) Product, P		

MULTIPLICATION-2

Signed Multiplication

- ❖ Observation
 - Here, if multiplicand(M) is negative or multiplier(Q) is negative, then product(P) is negative.
 - If M is negative and Q also is negative, then product is positive,
 - Hence $\text{sign}(\text{Product}, P) = \text{sign}(M) \text{ xor } \text{sign}(Q)$
- ❖ Hence
 - Convert M and Q to positive numbers with (n-1) bits Multiply positive numbers using shift-add method
 - Compute sign; depending on the sign, convert the product accordingly
- ❖ Alternatively,
 - Perform sign-extension on shift in shift-add method
 - Use Booth algorithm

MULTIPLICATION-2

Shift - Add Multiplication

1 1 0 1	(-3) Mul@plicand, M
X 0 1 0 1	(+5) Mul@plier, Q

1 1 1 1 1 0 1	
0 0 0 0 0 0	
1 1 1 0 1	
0 0 0 0	

1 1 1 1 0 0 0 1	(-15) Product, P

Consider signed numbers:

1101 → -3

0101 → +5

Product = -15 → 1111 0001

- ❖ Use Booth Algorithm for both positive and negative M & Q for the following reasons:
 - To reduce number of additions when sequence of 1's are in multiplier
 - To treat both positive and negative 2's complement n-bit operands uniformly

MULTIPLICATION - 2

Booth Multiplier T2 section-6.4

The method is a school trick:

When multiplying by 9

- Multiply by 10

- Subtract once

Ex: 234×9

234×10

2340

- 234

2106

Apply to binary:

When multiplying by 7

- Multiply by 8

- Subtract one Ex:

0101×0111

$0101 \times 100 -1$

1111011

000000

00000

0101

0100011 (+35) ✓

Method is known as Booth Recode

Reduces sequence of 1's to zeros and -1

Thus reduces number of additions



MULTIPLICATION-2

Booth's Multiplier

Booth Recoding Table:

Booth Recoding Table		
Bit i	bit i-1	Recoded Multiplier $\times$ Multiplicand
0	0	$0 \times M$
0	1	$+1 \times M$
1	0	$-1 \times M$
1	1	$0 \times M$

0 0 1 0 1 1 0 0 1 1 1 0 1 0 1 1 0 0

0 +1 -1 +1 0 -1 0 +1 0 0 -1 +1 -1 +1 0 -1 0 0

Booth recoding of a multiplier.

MULTIPLICATION-2

Booth's Multiplier

Booth Algorithm steps:

1. If the M or Q is negative then find the 2's complement of respective negative number.
2. Find the Recoded multiplier (Q') by referring Booth recording table.
3. Multiply with 2n-bit extended for each partial product P_i . Where -1 means 2's complement of Multiplicand (M).
4. Find the final Product P of 2n-bits.

MULTIPLICATION-2

Booth's Multiplier

Transitions are represented as:

0 0 $\rightarrow$ 0

0 1 $\rightarrow$ +1

1 0 $\rightarrow$ -1

1 1 $\rightarrow$ 0

Ex:

0 0 0 0 1 1 1 1 1 1 1 0 0 0 1 1 1 0 (16270)

MULTIPLICATION-2

Booth's Multiplier

Transitions are represented as:

0 0 $\rightarrow$ 0

0 1 $\rightarrow$ +1

1 0 $\rightarrow$ -1

1 1 $\rightarrow$ 0

Ex:

0 0 0 0 1 1 1 1 1 1 0 0 0 1 1 1 0 (16270)



0 0 0 1 0 0 0 0 0 0 -1 0 0 1 0 0 -1 0

No. of add & sub : 10 vs. 4

MULTIPLICATION-2

Booth's Multiplier

Transitions are represented as:

0 0 $\rightarrow$ 0

0 1 $\rightarrow$ +1

1 0 $\rightarrow$ -1

1 1 $\rightarrow$ 0

Ex:

0 0 0 0 1 1 1 1 1 1 0 0 0 1 1 1 0 (16270)



0 0 0 1 0 0 0 0 0 0 -1 0 0 1 0 0 -1 0

No. of add & sub : 10 vs. 4

Ex:

0 1 0 1 0 1 0 1 0

=>

+1 -1 +1 -1 +1 -1 +1 -1

Worst Case

MULTIPLICATION-2

Booth's Multiplier

Transitions are represented as:

0 0 $\rightarrow$ 0

0 1 $\rightarrow$ +1

1 0 $\rightarrow$ -1

1 1 $\rightarrow$ 0

Ex:

0 0 0 0 1 1 1 1 1 1 0 0 0 1 1 1 0 (16270)



0 0 0 1 0 0 0 0 0 0 -1 0 0 1 0 0 -1 0

No. of add & sub : 10 vs. 4

Ex:

0 1 0 1 0 1 0 1 0

=>

+1 -1 +1 -1 +1 -1 +1 -1

Worst Case

Ex:

0 1 1 1 1 1 1 1 0

=>

+1 0 0 0 0 0 0 -1

Best Case

MULTIPLICATION-2

The Booth Algorithm

- ❖ In general, in the Booth algorithm, -1 times the shifted multiplicand is selected when moving from 0 to 1, and $+1$ times the shifted multiplicand is selected when moving from 1 to 0, as the multiplier is scanned from right to left.
- ❖ Next figure illustrates the normal and the Booth algorithms for the example just discussed.

MULTIPLICATION-2

The Booth Algorithm

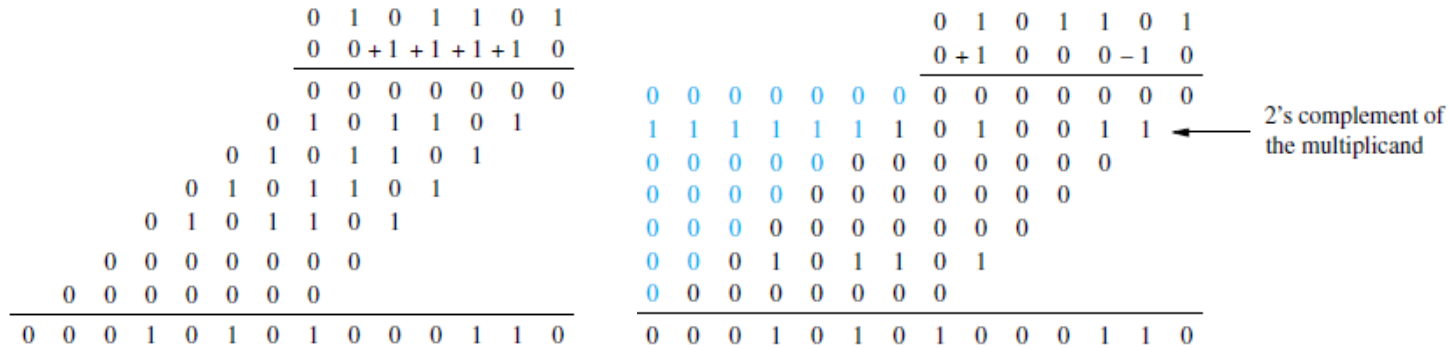


Figure Normal and Booth multiplication schemes.

MULTIPLICATION-2

The Booth Algorithm

- ❖ However, in the worst case—that of alternating 1s and 0s in the multiplier—each bit of the multiplier selects a summand.
- ❖ In fact, this results in more summands than if the Booth algorithm were not used.
- ❖ A 16-bit worst-case multiplier, an ordinary multiplier, and a good multiplier are shown in figure.

MULTIPLICATION-2

The Booth Algorithm

Worst-case multiplier	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
	+1	-1	+1	-1	+1	-1	+1	-1	+1	-1	+1	-1	+1	-1	+1	-1
Ordinary multiplier	1	1	0	0	0	1	0	1	1	0	1	1	1	1	0	0
	0	-1	0	0	+1	-1	+1	0	-1	+1	0	0	0	-1	0	0
Good multiplier	0	0	0	0	1	1	1	1	1	0	0	0	0	1	1	1
	0	0	0	+1	0	0	0	0	-1	0	0	0	+1	0	0	-1

Figure Booth recoded multipliers.

MULTIPLICATION-2

The Booth Algorithm

- ❖ The Booth algorithm has two attractive features.
- ❖ First, it handles both positive and negative multipliers uniformly.
- ❖ Second, it achieves some efficiency in the number of additions required when the multiplier has a few large blocks of 1s.

MULTIPLICATION-2

Booth's Multiplier

Transitions are represented as:

0 0 $\rightarrow$ 0

0 1 $\rightarrow$ +1

1 0 $\rightarrow$ -1

1 1 $\rightarrow$ 0

Ex:

0 0 0 0 1 1 1 1 1 1 1 0 0 0 1 1 1 0



0 0 0 1 0 0 0 0 0 0 -1 0 0 1 0 0 -1 0 (16270)

No. of add & sub : 10 vs. 4

Depending on Bit(i+1) and Bit(i) of the Multiplier, Q

following operation can be defined on A:

0 0 $\rightarrow$ Shift Right (0 x M)

1 1 $\rightarrow$ Add M (+1 x M)

1 0 $\rightarrow$ Subtract (2's complement) M (-1 x M)

1 1 $\rightarrow$ Shift Right (0 x M)

Ex:

0 1 0 1 0 1 0 1 0

=>

+1 -1 +1 -1 +1 -1 +1 -1

Ex:

0 1 1 1 1 1 1 1 0

=>

+1 0 0 0 0 0 0 -1

Worst Case

Best Case

MULTIPLICATION-2

Booth's Multiplier

Booth algorithm using - shift and add

$Q_0 Q_{-1}$

0 0 - no add, shift right with sign extension

0 1 - Add M to a(+1 X M), right shift

1 0 - subtract M to A(-1 X M) $A=A - M$, right shift

1 1 - No add, shift right with sign extension

MULTIPLICATION - 2

Booth Multiplier

M			
1 1 0 1			1101 => -3 Multiplicand, M
0 0 0 0			1011 => -5 Multiplier, Q
A	Q	Q ⁻¹	
0 0 1 1	1 0 1 1	0	Q ₀ Q ⁻¹ = 10, Add 2's Comp of M to A
0 0 0 1	1 1 0 <u>1</u>	<u>1</u>	Shi< Right with Sign Extension
0 0 0 0	1 1 1 <u>0</u>	<u>1</u>	Q ₀ Q ⁻¹ =11, Shi< Right with Sign Ext
1 1 0 1	1 1 1 0	1	Q ₀ Q ⁻¹ =01, Add M to A
1 1 1 0	1 1 1 1	0	Shift Right with Sign Extension
0 0 0 1	1 1 1 1	0	Q ₀ Q ⁻¹ =10, Add 2's Comp of M to A Shift
0 0 0 0	1 1 1 1	1	Right with Sign Extension
(+15) Product, P			



Depending on Q₀ and Q⁻¹

Q₀ Q⁻¹

- 00 No Add, Shi< Right with Sign Extn. (0 x M) 0
- 1 Add M to A (+1 x M)
- 10 Subtract (2's complement) M to A (-1 x M) 1
- 11 No Add, Shi< Right with Sign Extn. (0 x M)

MULTIPLICATION-2

Booth's Multiplier

❖ **Points to be noted :**

- Handles both positive and negative multipliers uniformly.
- Achieves efficiency in the number of additions required when the multiplier has a few large blocks of 1's.(Skipping of 1's)
- In worst case, the speed of doing multiplication with the Booth algorithm will be same as the normal algorithm.

This is also known as **Radix-2 Coding technique**.

MULTIPLICATION-2

Think about it

- Apply Booth algorithm to the following:
 1. $+M \times -Q$
 2. $-M \times +Q$
 3. $+M \times +Q$
- Implement Booth algorithm using sequential repeated addition circuit or using Array Multiplier circuit.
- Instead of Shift Right Accumulator can we have Shift Left Multiplicand?

MULTIPLICATION-2

Fast Multiplication T2 section 6.5

- ❖ Let us now describe two techniques for speeding up the multiplication operation.
- ❖ The first technique guarantees that the maximum number of summands (versions of the multiplicand) that must be added is $n/2$ for n bit operands.
- ❖ The second technique leads to adding the summands in parallel.

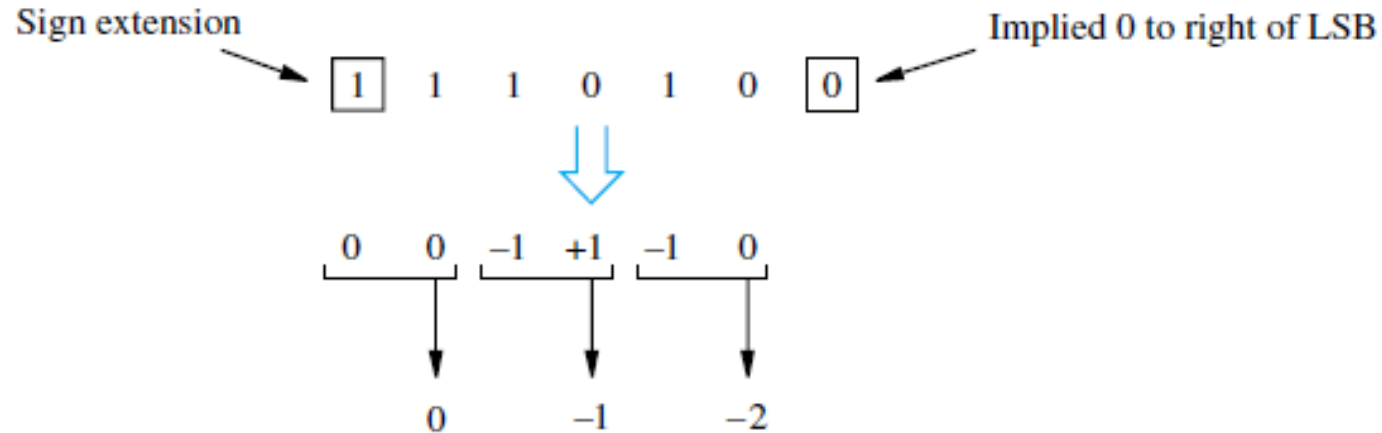
MULTIPLICATION-2

Bit-Pair Recoding of Multipliers

- ❖ A technique called bit-pair recoding of the multiplier results in using at most one summand for each pair of bits in the multiplier.
- ❖ It is derived directly from the Booth algorithm.
- ❖ Group the Booth-recoded multiplier bits in pairs, and observe the following.
- ❖ The pair $(+1 -1)$ is equivalent to the pair $(0 +1)$.
- ❖ That is, instead of adding -1 times the multiplicand M at shift position i to $+1 \times M$ at position $i + 1$, the same result is obtained by adding $+1 \times M$ at position i .
- ❖ Other examples are: $(+1 0)$ is equivalent to $(0 +2)$, $(-1 +1)$ is equivalent to $(0 -1)$, and so on.

MULTIPLICATION-2

Bit-Pair Recoding of Multipliers



(a) Example of bit-pair recoding derived from Booth recoding

MULTIPLICATION-2

Bit-Pair Recoding of Multipliers

Multiplier bit-pair		Multiplier bit on the right $i - 1$	Multiplicand selected at position i
$i + 1$	i		
0	0	0	$0 \times M$
0	0	1	$+ 1 \times M$
0	1	0	$+ 1 \times M$
0	1	1	$+ 2 \times M$
1	0	0	$- 2 \times M$
1	0	1	$- 1 \times M$
1	1	0	$- 1 \times M$
1	1	1	$0 \times M$

(b) Table of multiplicand selection decisions

MULTIPLICATION-2

Bit-Pair Recoding of Multipliers

- ❖ Thus, if the Booth-recoded multiplier is examined two bits at a time, starting from the right, it can be rewritten in a form that requires at most one version of the multiplicand to be added to the partial product for each pair of multiplier bits.
- ❖ Figure (a) shows an example of bit-pair recoding of the previous multiplier and figure (b) shows a table of the multiplicand selection decisions for all possibilities.
- ❖ The multiplication operation in the previous example is shown in the next figure as it would be computed using bit-pair recoding of the multiplier.

MULTIPLICATION-2

Bit-Pair Recoding of Multipliers

$$\begin{array}{r}
 01101 \quad (+13) \\
 \times 11010 \quad (-6) \\
 \hline
 \end{array}$$

↓

$$\begin{array}{r}
 01101 \\
 0-1+1-10 \\
 \hline
 00000 \quad 00000 \\
 11111 \quad 10011 \\
 00001 \quad 1101 \\
 11100 \quad 11 \\
 00000 \\
 \hline
 1110110010 \quad (-78)
 \end{array}$$

$$\begin{array}{r}
 01101 \\
 0 \quad -1 \quad -2 \\
 \hline
 11111 \quad 00110 \\
 11110 \quad 011 \\
 00000 \\
 \hline
 1110110010
 \end{array}$$

Figure Multiplication requiring only $n/2$ summands.

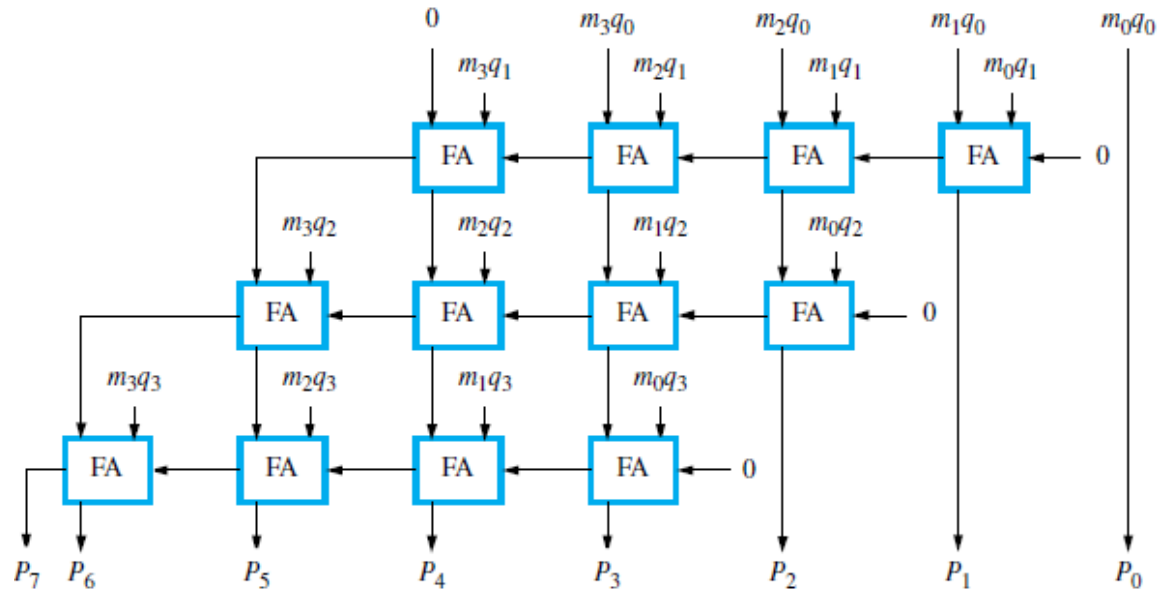
MULTIPLICATION-2

Carry-Save Addition of Summands

- ❖ Multiplication requires the addition of several summands.
- ❖ A technique called **carry-save addition (CSA)** can be used to speed up the process.
- ❖ Consider the 4×4 multiplication array shown in figure (a).
- ❖ This structure is in the form of the array, in which the first row consists of just the AND gates that produce the four inputs m_3q_0 , m_2q_0 , m_1q_0 , and m_0q_0 .

MULTIPLICATION-2

Carry-Save Addition of Summands



(a) Ripple-carry array

MULTIPLICATION-2

Wallace Tree Multiplier

Carry Save Addition

Carry Save Example

Sum Carry	+		0	1	1	1
			1	0	1	1
		0	1	1	1	0
			1	0	1	0
		0	1	1	1	0
		1	1	0	0	0

Carry Save Addition

Given three n -bit numbers

- 1 Compute two n -bit numbers (n sum bits and n carry bits).
- 2 Add $n - 1$ leftmost sum bits to n carry bits obtaining $n + 1$ bit result and attach remaining sum bit as LSB.

MULTIPLICATION-2

Wallace Tree Multiplier

Carry Save Adder Design

Example of Carry Save Addition

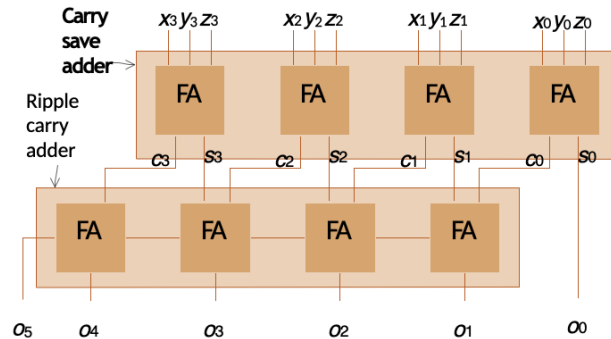
X: 1 0 0 1 1	X: 1 0 0 1 1
Y: + 1 1 0 0 1	Y: + 1 1 0 0 1
Z: + 0 1 0 1 1	Z: + 0 1 0 1 1
C: 1 1 0 1 1	S: 0 0 0 0 1

X: 1 0 0 1 1
Y: + 1 1 0 0 1
Z: + 0 1 0 1 1
S: 0 0 0 0 1
C: 1 1 0 1 1
Sum: 1 1 0 1 1 1

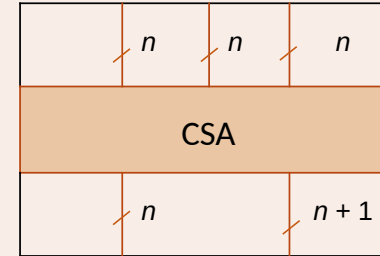
MULTIPLICATION-2

Wallace Tree Multiplier

Logic for carry save addition of three numbers:



Basic Carry Save Adder



Contains n full adders.

Inputs:

- Three n -bit numbers

Outputs:

- One n -bit number
- One $(n + 1)$ -bit number (whose LSB is 0)

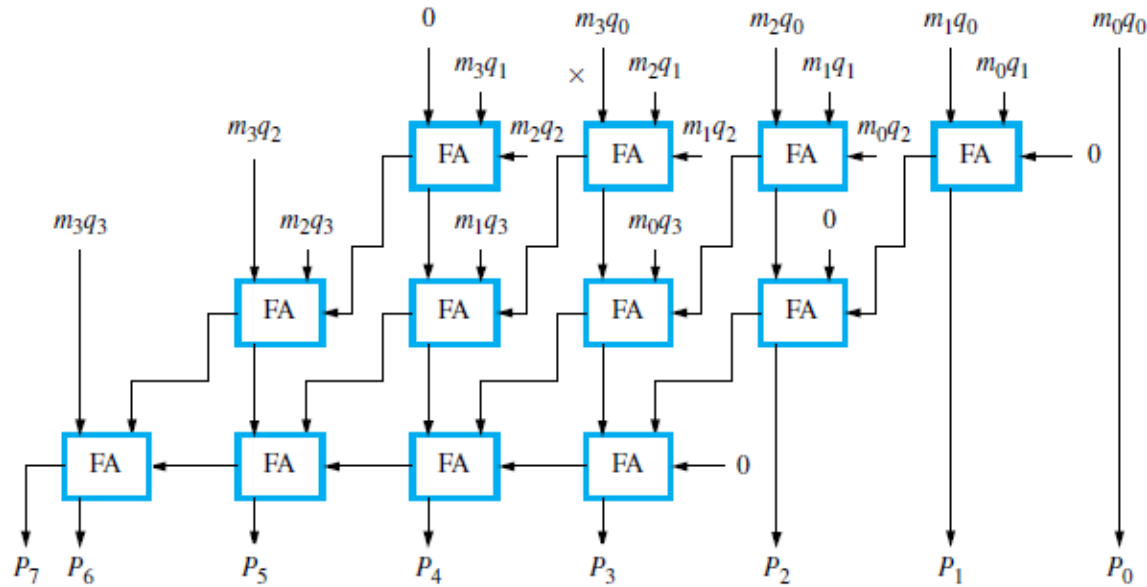
MULTIPLICATION-2

Carry-Save Addition of Summands

- ❖ Instead of letting the carries ripple along the rows, they can be “**saved**” and introduced into the next row, at the correct weighted positions, as shown in figure (b).
- ❖ This frees up an input to each of three full adders in the first row.
- ❖ These inputs can be used to introduce the third summand bits m_2q_2 , m_1q_2 , and m_0q_2 .

MULTIPLICATION-2

Carry-Save Addition of Summands



(b) Carry-save array

MULTIPLICATION-2

Carry-Save Addition of Summands

- ❖ Now, two inputs of each of three full adders in the second row are fed by the sum and carry outputs from the first row.
- ❖ The third input is used to introduce the bits m_2q_3 , m_1q_3 , and m_0q_3 of the fourth summand.
- ❖ The high-order bits m_3q_2 and m_3q_3 of the third and fourth summands are introduced into the remaining free full-adder inputs at the left end in the second and third rows.

MULTIPLICATION-2

Carry-Save Addition of Summands

- ❖ The saved carry bits and the sum bits from the second row are now added in the third row, which is a ripple-carry adder, to produce the final product bits.
- ❖ **The delay through the carry-save array is somewhat less than the delay through the ripple-carry array.**
- ❖ This is because the S and C vector outputs from each row are produced in parallel in one full-adder delay.

MULTIPLICATION-2

Summand Addition Tree Using 3-2 Reducers

- ❖ A more significant reduction in delay can be achieved when dealing with longer operands than those considered in the previous example.
- ❖ **We can group the summands in threes and perform carry-save addition on each of these groups in parallel to generate a set of S and C vectors in one full-adder (or simply an adder) delay.**
- ❖ Next, we group all the S and C vectors into threes, and perform carry-save addition on them, generating a further set of S and C vectors in one more adder delay.

MULTIPLICATION-2

Summand Addition Tree Using 3-2 Reducers

- ❖ We continue with this process until there are only two vectors remaining.
- ❖ The adder at each bit position of the three summands is called a **3-2 reducer**, and the logic circuit structure that reduces a number of summands to two is called a **CSA/Wallace tree**.
- ❖ The final two **S** and **C** vectors can be added in a carry-lookahead adder to produce the desired product.

MULTIPLICATION-2

Summand Addition Tree Using 3-2 Reducers

- ❖ Three levels of carry-save addition are performed, as shown schematically in figure (c).
- ❖ This figure shows that the final two vectors S_4 and C_4 are available in three adder delays after the six input summands are applied to level 1.
- ❖ The final regular addition operation on S_4 and C_4 , which produces the product, can be done with a carry-lookahead adder.

MULTIPLICATION-2

Summand Addition Tree Using 3-2 Reducers

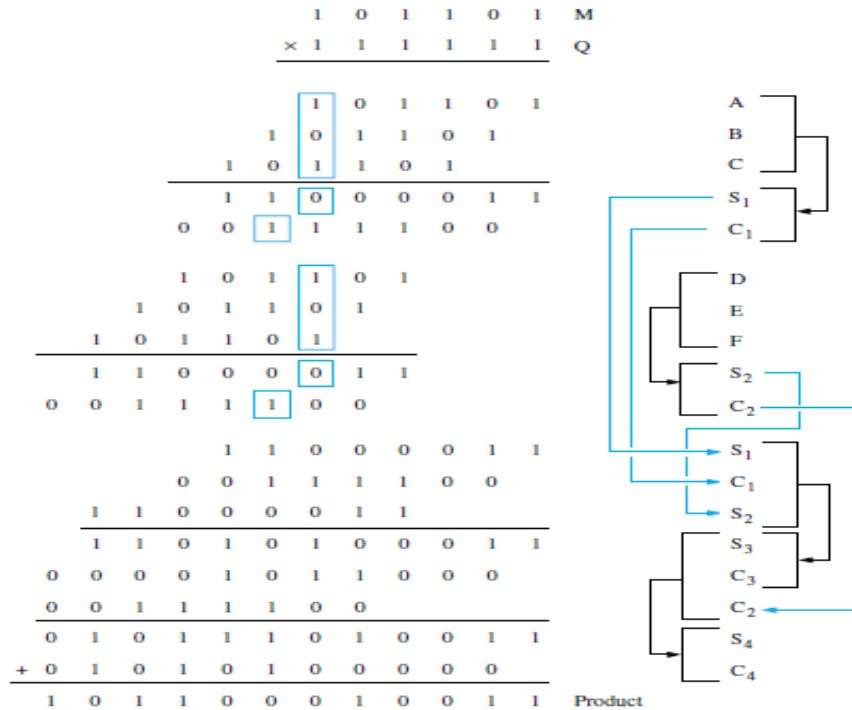


Figure (b) The multiplication performed using carry-save addition.

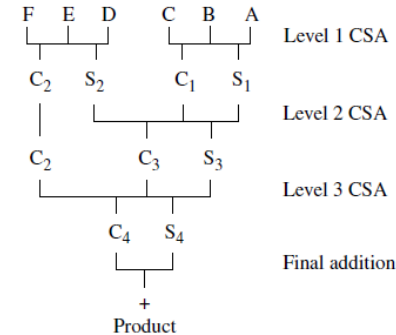


Figure (c) Schematic representation of the carry-save addition operations

MULTIPLICATION-2

Think About It

- ❖ For $n \times n$ array multiplication we need $6(n-1)$ delay
- ❖ For carry save addition method the $1.7 \log_2 k - 1.7$ number of CSA levels of CSA steps are needed to reduce K summands to 2 vectors, which when added produces the desired result.

MULTIPLICATION-2

Think About It

Develop the derivation for the formula $1.7 \log_2 k - 1.7$ for the number of carry-save addition steps needed to reduce k summands to two vectors. (This formula is stated without derivation in Section 6.5.2.)

The number of reduction steps n to reduce k summands to 2 is given by $k(2/3)^n = 2$, because each step reduces 3 summands to 2. Then we have:

$$\begin{aligned}
 \log_2 k + n(\log_2 2 - \log_2 3) &= \log_2 2 \\
 \log_2 k &= 1 + n(\log_2 3 - \log_2 2) \\
 &= 1 + n(1.59 - 1) \\
 n &= \frac{(\log_2 k) - 1}{0.59} \\
 &= 1.7 \log_2 k - 1.7
 \end{aligned}$$

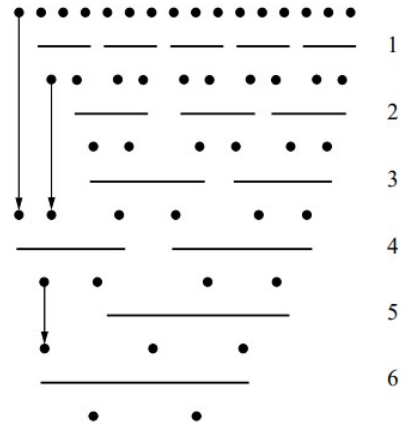
This answer is only an approximation because the number of summands is not a multiple of 3 in each reduction step.

MULTIPLICATION-2

Think About It

- How many CSA levels are needed to reduce 16 summands to 2 using Carry save addition method?

(a) Six CSA levels are needed:

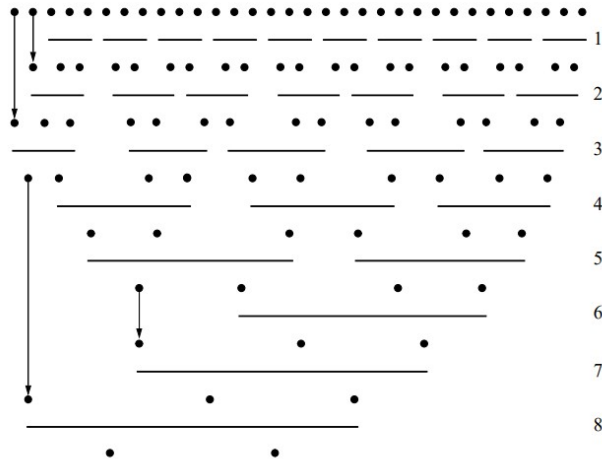


MULTIPLICATION-2

Think About It

2. How many CSA levels are needed to reduce 32 summands to 2 using Carry save addition method?

(b) Eight CSA levels are needed:



If we apply formula $1.7 \log K - 1.7$

Then,

The approximation gives 5.1 and 6.8 CSA levels, compared to 6 and 8

MULTIPLICATION-2

Think About It

2. How many CSA levels are needed to reduce 32 summands to 2 using Carry save addition method?

The gate delay for 32 X 32 multiplication is:

There are 8 levels required

One AND gate delay for multiplication

8 levels uses 16 gate delays assuming each full adder takes 2 gate delays

12 gate delays for last step of fast adder which CLA

Total= $16+12+1=29$ gate delays

IF RCA is used= $6(n-1)-1=185$ gate delays

In Booth's multiplication algorithm, when the pair of bits $(Q_0, Q_{-1}) = (1, 0)$, the operation performed on the accumulator A is:

- A. Add multiplicand (M)
- B. Subtract multiplicand (M)
- C. No operation (shift only)
- D. Add twice multiplicand ($M \times 2$)

Bit-pair recoding in multiplication reduces:

- A. Number of partial products
- B. Hardware complexity
- C. Number of control signals
- D. Both A and B

In Booth's multiplication algorithm, when the pair of bits $(Q_0, Q_{-1}) = (1, 0)$, the operation performed on the accumulator A is:

- A. Add multiplicand (M)
- B. Subtract multiplicand (M)
- C. No operation (shift only)
- D. Add twice multiplicand ($M \times 2$)

Answer: B — *Booth rule: $(1\ 0) \rightarrow -1 \times M \rightarrow$ Subtract multiplicand.*

Bit-pair recoding in multiplication reduces:

- A. Number of partial products
- B. Hardware complexity
- C. Number of control signals
- D. Both A and B

Answer: D — *Bit-pair recoding halves the number of partial products and simplifies control logic.*

If three 4-bit numbers are added using a CSA, what is the bit-width of the carry output after alignment (left shift)?

- A. 4 bits
- B. 5 bits
- C. 8 bits
- D. 3 bits

For $k = 16$ summands, the number of CSA levels needed to reduce them to two vectors:

- A. 12
- B. 10
- C. 8
- D. 6

If three 4-bit numbers are added using a CSA, what is the bit-width of the carry output after alignment (left shift)?

- A. 4 bits
- B. 5 bits
- C. 8 bits
- D. 3 bits

Answer: B. 5 bits

For $k = 16$ summands, the number of CSA levels needed to reduce them to two vectors

- A. 12
- B. 10
- C. 8
- D. 6

Answer: D



THANK YOU

Team DDCO

Department of Computer Science & Engineering



DIGITAL DESIGN & COMPUTER ORGANISATION

Division

Team DDCO

Department of Computer Science &
Engineering

DIGITAL DESIGN & COMPUTER ORGANISATION

Division

Team DDCO.

Department of Computer Science & Engineering

DIVISION

Course Outline

- ❖ Digital Design
 - Combinational logic design
 - Sequential logic design
 - Divider
- ❖ Computer Organisation
 - Architecture (microprocessor instruction set)
 - Microarchitecture (microprocessor operation)

Concepts covered

- ❖ Binary Integer Division

DIVISION

Long Hand Method

- ❖ Manual method of division of decimal number
- ❖ Binary division also can be performed in a similar way
- ❖ Repetitive Subtraction: Mark 1 for success and 0 for failure

DIVISION- T2 section 6.6

Long Hand Method

- ❖ Manual method of division of decimal number
- ❖ Binary division also can be performed in a similar way
- ❖ Repetitive Subtraction: Mark 1 for success and 0 for failure

(13) Divisor, M. $\Rightarrow$ 1101

(274) Dividend, Q $\Rightarrow$ 100010010

$$\begin{array}{r} 021 \\ 13 \overline{) 274} \\ \underline{-26} \\ 14 \\ \underline{-13} \\ 1 \end{array}$$

DIVISION

Long Hand Method

- ❖ Manual method of division of decimal number
- ❖ Binary division also can be performed in a similar way
- ❖ Repetitive Subtraction: Mark 1 for success and 0 for failure

(13) Divisor, M. $\Rightarrow$ 1101

(274) Dividend, Q $\Rightarrow$ 100010010

$$\begin{array}{r} 021 \\ 13 \overline{) 274} \\ \underline{-26} \end{array}$$

14

-13

1

Quotient¹ = 21 $\Rightarrow$ 10101; Remainder = 1

$$\begin{array}{r} 000010101 \\ 1101 \overline{) 100010010} \\ \underline{1101} \downarrow \downarrow \downarrow \downarrow \\ 10000 \\ \underline{1101} \downarrow \downarrow \downarrow \downarrow \\ 1110 \\ \underline{1101} \downarrow \downarrow \downarrow \downarrow \\ 1 \end{array}$$

DIVISION

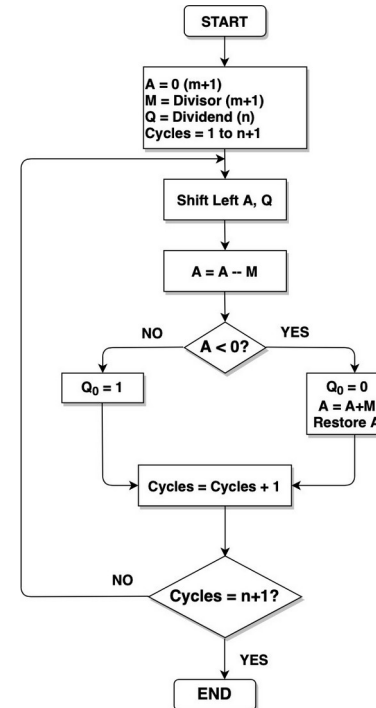
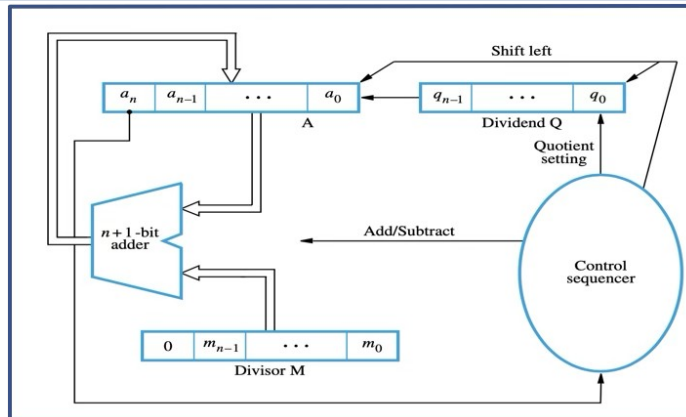
Restoring Division

1. Initialize Registers for two n-bit division operation, register M with Divisor, n+1 (i.e., sign bit) bits register A with Zero (0), register Q with Dividend(N bit)
2. Repeat n times
 - i. Left shift Q to A by one binary position.
 - ii. Subtract M from A and Place the answer back in A i.e., $A = A - M$.
 - iii. if Sign bit of A is 1 then set $q_0 = 0$ and Add M to A and Place the answer back in A i.e., $A = A + M$ (Restore) else if Sign bit of A is 0 then set $q_0 = 1$ and no restore.
3. After n-cycles, A will contain Remainder and Q contains Quotient.

DIVISION

Restoring Division

- ❖ We Require:
- Accumulator register – A ($m+1$) bits (Remainder)
 - Dividend register – Q (Quotient) n -bits
 - Divisor register – M ($m+1$ bit $\Rightarrow$ Sign)
 - $n+1$ -bit Adder
 - Control signals for Shift left and Add/Sub



DIVISION

Restoring Division

	M	
	00011	
0	0000	1000
S	A	Q
0	0001	000?
1	1101	000?
1	1101	0000
0	0001	0000

Consider: Divisor => 00011
 2's Comp=> 11101
 Dividend => 1000

Shift Left
 Sub M (2's comp)
 S = 1, Set Q₀ to 0
 Restore A

0	0010	0000
S	A	Q
0	0100	000?
1	1101	000?
0	0001	0001
0	0010	001?
1	1101	001?
1	1111	0010
0	0010	0010

Shift Left
 Sub M (2's comp)
 S = 0, Set Q₀ to 1
 Shift Left
 Sub M (2's comp)
 S = 1, Set Q₀ to 0
 Restore A
 Remainder=2; Quotient=2
 No. of Add/Sub performed: 7

No. of Add/Sub performed: 7

DIVISION

Restoring Division

❖ Observations:

- This method is used for unsigned binary numbers division or positive signed binary numbers division
- There are significant number of addition and subtraction required due to restoration process.
- **Avoid the need to restore after an unsuccessful subtraction.**
- If A is positive, we shift left and subtract $M \Rightarrow A-M$
- If A is negative, we restore A by performing $A+M$ and then shift left and subtract $M \Rightarrow (A+M)-M = A$
- $A+M$, represents shift left and add M, when A is negative.
- This leads to **Non-Restoring division** algorithm.

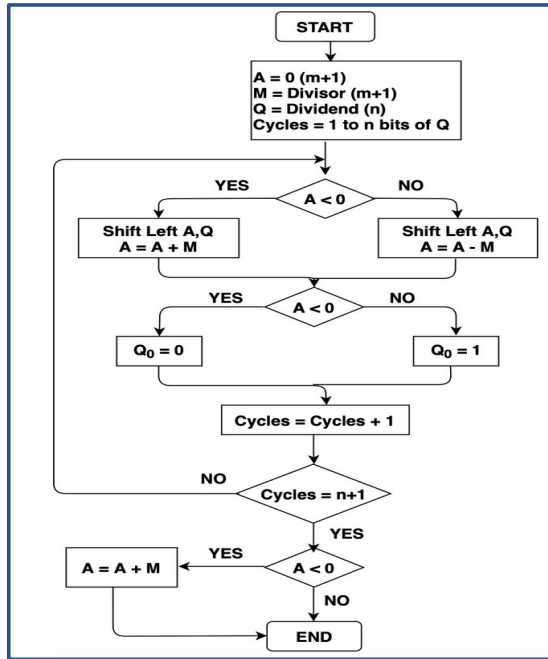
DIVISION

Non - Restoring Division

1. Initialize Registers for two n-bit division operation, register M with Divisor(n+1), register Q with Dividend n bits, register A with Zero (0) n+1 bit- including sign bit.
2. Repeat n times
 - i. Left shift Q to A by one binary position
 - ii. if Sign bit of A is 0 then subtract M from A and Place the answer back in A i.e., $A=A-M$ else if Sign bit of A is 1 then add M to A and Place the answer back in A i.e., $A=A+M$.
 - iii. If Sign bit of A is 0 then set $q_0=1$ else set $q_0=0$
3. After n-cycles, if Sign bit of A is 1 then add M to A and Place the answer back in A i.e., $A=A+M$, A will contains Remainder and Q contains Quotient.

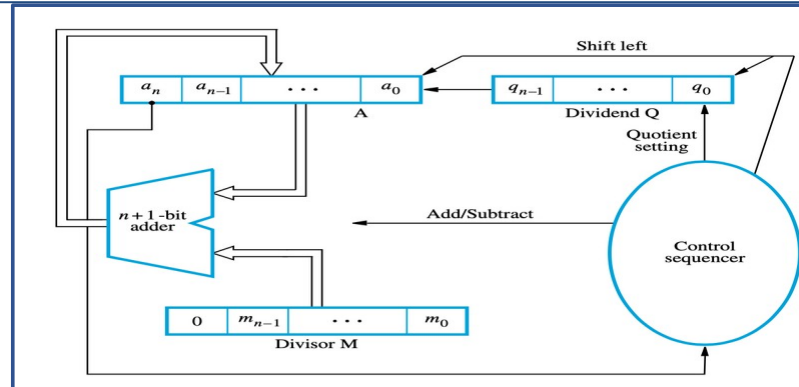
DIVISION

Non - Restoring Division



❖ We Require:

- Accumulator register - A (m+1) bits (Remainder)
- Dividend register - Q (Quotient) n-bits
- Divisor register - M (m+1 bit => Sign)
- n+1-bit Adder
- Control signals for Shift left and Add/Sub



DIVISION

Non - Restoring Division

M			Consider:
0 0 0 1 1			Divisor => 00011
0	0 0 0 0	1 0 0 0	2's Comp=> 11101
S	A	Q	Dividend => 1000
0	0 0 0 1	0 0 0 ?	Shift Left
1	1 1 0 1	0 0 0 ?	Sub M (2's comp)
1	1 1 1 0	0 0 0 0	S = 1, Set Q ₀ to 0
1	1 1 0 0	0 0 0 ?	Shift Left
0	0 0 1 1	0 0 0 ?	Add M
1	1 1 1 1	0 0 0 0	S=1, Set Q ₀ to 0
1	1 1 1 0	0 0 0 ?	Shift Left
0	0 0 1 1	0 0 0 ?	Add M
0	0 0 0 1	0 0 0 1	S = 0, Set Q ₀ to 1
0	0 0 1 0	0 0 1 ?	Shift Left
1	1 1 0 1	0 0 1 ?	Sub M (2's comp)
1	1 1 1 1	0 0 1 0	S = 1, Set Q ₀ to 0

1	1 1 1 1	0 0 1 0
S	A	Q
1	1 1 1 1	0 0 1 0
0	0 0 1 1	0 0 1 0
0	0 0 1 0	0 0 1 0
Restore A		
Remainder=2; Quotient=2		

No. of Add/Sub performed: 5

DIVISION

Non - Restoring Division

- ❖ Points to be noted :
 - Both Restoration and Non Restoration Division works for unsigned and positive signed numbers.
 - For signed numbers, convert the numbers to positive and adjust the sign later.
 - Simple solution Sign of the result will be XOR function of signs of Divisor and Dividend.

- ❖ Ex:
 - **$7 \div 2 = 3$ and the remainder is 1**
 - **$-7 \div 2 = -3$ and the remainder is - 1**

DIVISION

Think about it

- ❖ Compare the number of addition/subtraction for the following operation using restoration and non-restoration methods:
 - $111110 \div 0111$
 - $1000010 \div 0101$
- ❖ For n-bits what will be the worst case in terms of number of additions and subtractions for restoration and non-restoration division?

DIVISION QUESTIONS

- ❖ Perform the division of following numbers using restoring division algorithm: Dividend = 7 = 0111 Divisor = 6 = 0110
- ❖ Apply non-restoring division algorithm on following operands: Dividend = 11 = 1011 Divisor = 4 = 0100

DIVISION

MCQ

In the restoring division algorithm, if after subtraction the sign bit of register A is 1, then which of the following operations is performed next?

- A. Left shift Q to A by one position
- B. Subtract M again from A
- C. Add M to A (restore) and set $q_0 = 0$
- D. Add M to A (restore) and set $q_0 = 1$

DIVISION

MCQ

In the restoring division algorithm, if after subtraction the sign bit of register A is 1, then which of the following operations is performed next?

- A. Left shift Q to A by one position
- B. Subtract M again from A
- C. Add M to A (restore) and set $q_0 = 0$
- D. Add M to A (restore) and set $q_0 = 1$

Answer: C

Explanation: When the remainder becomes negative (sign bit = 1), restoration is performed by adding back the divisor ($A = A + M$) and the quotient bit $q_0 = 0$

DIVISION

MCQ

Whats is the value of accumulator and register Q at end of first cycle for restoring division of 1110/0011.

- a. Accumulator= 00001 Q=1100
- b. Accumulator= 11001 Q=1101
- c. Accumulator= 00100 Q=1100
- d. Accumulator= 00001 Q=1001

In both restoring and non-restoring division, the quotient bit (q_0) depends on:

- A. The sign of M
- B. The sign of A
- C. The sign of Q
- D. Carry from adder

DIVISION

MCQ

is the value of accumulator and register Q at end of first cycle for restoring division of 1110/0011.

- a. Accumulator= 00001 Q=1100
- b. Accumulator= 11001 Q=1101
- c. Accumulator= 00100 Q=1100
- d. Accumulator= 00001 Q=1001

ans: a

In both restoring and non-restoring division, the quotient bit (q_0) depends on:

- A. The sign of M
- B. The sign of A
- C. The sign of Q
- D. Carry from adder

Answer: B

Explanation: Quotient bit is determined by the sign bit of A after each operation.



THANK YOU

Team DDCO

Department of Computer Science & Engineering



DIGITAL DESIGN & COMPUTER ORGANISATION

Floating Point

Team DDCO

Department of Computer Science &
Engineering

DIGITAL DESIGN & COMPUTER ORGANISATION

Floating Point

Department of Computer Science & Engineering

Floating Point operations- T2 section 6.7.1

Course Outline

- ❖ Digital Design
 - Combinational logic design
 - Sequential logic design
 - Floating Point
- ❖ Computer Organisation
 - Architecture (microprocessor instruction set)
 - Microarchitecture (microprocessor operation)

Concepts covered

- Floating Point Representation

Floating Point

Not Just Integers

- ❖ Real numbers can be represented using:
 - Fixed point
 - Floating point
- ❖ Fixed point notation is where the decimal point is fixed and numbers to the right of decimal point are the fraction portion and to the left is the integer portion.
 - Limited by the digits used
 - Not suitable to represent very small or very large numbers
- ❖ Programming languages support fraction called **floating point** numbers
 - Example: 3.14159265... (π); 2.71828... (e)
 - Data type used float , double

Number Systems

Fixed-Point Number Systems

- ❖ Signed fixed-point numbers can use either two's complement or sign/magnitude notation.
- ❖ Figure shows the fixed-point representation of -2.375 using both notations with four integer and four fraction bits.

(a) 0010.0110

(b) 1010.0110

(c) 1101.1010

**Figure Fixed-point
representation of -2.375 :**

**(a) absolute value, (b) sign and
magnitude, (c) two's complement**

Floating Point

Fixed Point Example

- ❖ Represent 6.75 using 4 integer bits and 4 fraction bits:
 - $6 \Rightarrow 0110 (2^2 + 2^1)$
 - $0.75 \Rightarrow 0.1100 (2^{-1} + 2^{-2})$
 - $6.75 \Rightarrow 0110.1100$

- ❖ Here binary point is implied and the number of bits used is decided beforehand
 - Fixed point
 - Floating point

- ❖ Represent -7.5 using 4 integer and 4 fraction bits
 - $+7.5 \Rightarrow 0111.1000$
 - 2's complement $-7.5 \Rightarrow 1000.1000$

Floating Point

Fixed Point Example

- ❖ Perform the following operation: $7.5 - 0.625 \Rightarrow 7.5 + (-0.625)$
 - $7.5 \Rightarrow 0111.1000$
 - $-0.625 \Rightarrow 111.0110$ (2's complement)
 - $0111.1000 + 1111.0110 = 0110.1110$ (6.875)

- ❖ The range and accuracy is very limited.
 - Ex: $8.9375 + 8.3125 = 17.2495$
 - $8.9375 \Rightarrow 1000.1111$
 - $8.3125 \Rightarrow 1000.0101$
 - Add: 0001.0100 (1.25) which is the result of limited range and limited accuracy

- ❖ How to increase the range and improve the accuracy?
 - Go for Floating Point Representation

Floating Point

Not Just Integers

- ❖ Floating point notation is used to represent real numbers which are from small to large numbers
- ❖ We use scientific notation to represent these numbers
 - $\pm d.f_1f_2f_3... \times 10^{\pm e_1e_2}$
 - $\pm M \times B^{\pm E}$
- ❖ This representation is to include very small numbers like 1.0×10^{-23} and very larger numbers like 9.546×10^{12}
- ❖ Floating point numbers should be **normalized**
 - Use one non-zero digit as integer
 - In decimal it will be from 1 to 9
 - In binary this should be 1
 - Ex:
 - Normalised floating point: 2.234×10^3 or 1.101×2^{-4}
 - Non-normalized floating point: 0.0234×10^5 or 110.1×2^{-6}

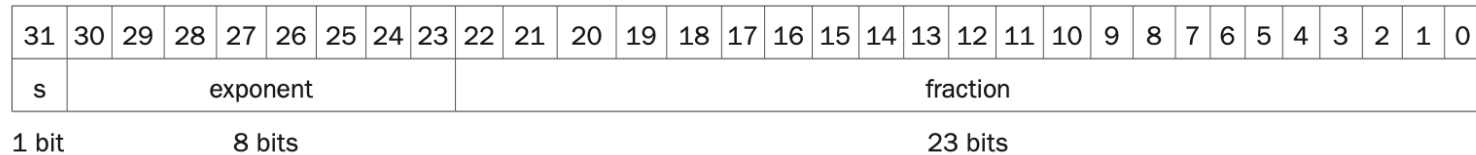
Floating Point

IEEE 754-2008 Standard

- ❖ IEEE Standard defines structure of floating point number representation
- ❖ Developed in response to divergence of representations and arithmetic operations
 - Portability issues for scientific code
 - Universally adopted
- ❖ Defines four representations:
 - Single Precision (32-bits)
 - Double Precision (64-bits)
 - Extended Double Precision 10 bytes (80-bits)
 - Quadruple Precision 16 bytes(128-bits)
- ❖ Real Number is represented in IEEE 754-2008 standard as three parts:
 - Sign bit
 - Exponent bits
 - Mantissa bits or Significand bits

Floating Point

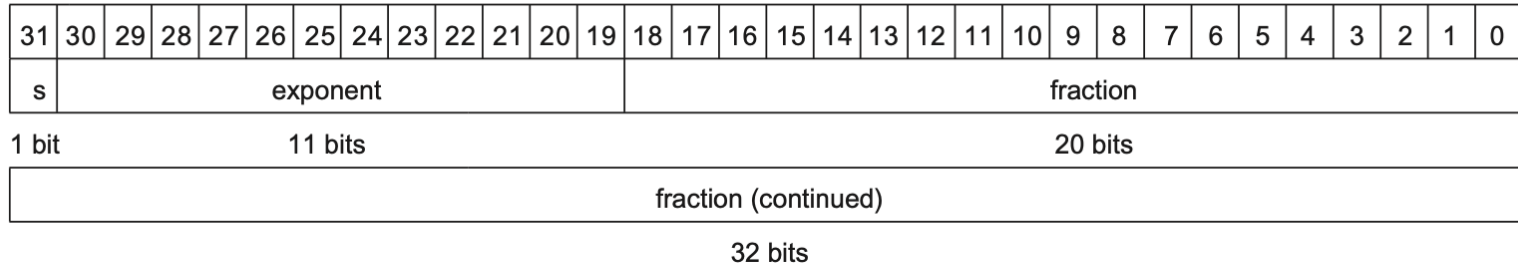
IEEE 754-2008 Standard (Single Precision)



- ❖ Sign bit 0 indicates positive and 1 indicates negative number
- ❖ Mantissa represents fraction and signifies accuracy of the number
- ❖ Exponent represents range of the numbers that shall be represented
- ❖ General form: $\pm 1.\text{Mantissa} \times 2^{\text{Exponent}}$
- ❖ For Single precision (32-bits) representation:
 - Biased Exponent is 8-bits
 - Mantissa is 23 bits

Floating Point

IEEE 754-2008 Standard (Double Precision)



- ❖ Sign bit 0 indicates positive and 1 indicates negative number
- ❖ Mantissa represents fraction and signifies accuracy of the number
- ❖ Exponent represents range of the numbers that shall be represented
- ❖ General form: $\pm 1.\text{Mantissa} \times 2^{\text{Exponent}}$
- ❖ For Double precision (64-bits) representation:
 - Biased Exponent is 11-bits
 - Mantissa is 52 bits

Floating Point

Biased Exponent

- ❖ Biased Exponent, $BE = \text{Bias} + \text{Exponent}$
- ❖ Actual Exponent, $E = \text{Biased Exponent} - \text{Bias}$
- ❖ Recall for Single precision Biased Exponent is 8-bits (Range: 0 to 255)
- ❖ $BE = 0$ and $BE = 255$ are reserved for special use
- ❖ So, $BE = 1$ to 254 are used for normalized floating point numbers.
- ❖ $\text{Bias} = 127 \Rightarrow (2^{n-1} - 1)$
- ❖ Therefore Range of Actual exponent that could be represented is:
 - $\text{Min} = 1 - 127 = -126$
 - $\text{Max} = 254 - 127 = 127$
 - So range is from -126 to $+127$
- ❖ FP Representation:

$$\blacksquare N = (-1)^s * (1+.M) * 2^{(BE-\text{Bias})}$$

Bias = 127 for SP

Bias = 1023 for DP

Floating Point

FP Example

What is the value of the following number:

0	100 0011 0	110 0100 0000 0000 0000 0000
---	------------	------------------------------

In Hexadecimal this is represented as 0x43640000

Solution:

Sign = 0; Positive number

Biased Exponent = $(1000\ 0110)_2 = 134$;

Actual Exponent = $134 - 127 = 7$

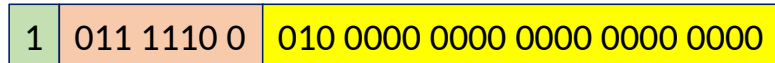
Mantissa = $(1.1100\ 10...000)_2 = 1.78125$ (1. is implicit)

So, the value of the decimal = $1.1100100 \times 2^7 = 11100100 =$ 228

Floating Point

FP Example

What is the value of the following number:



In Hexadecimal this is represented as 0xBE200000

Solution:

Sign = 1;

Biased Exponent = $(0111\ 1100)_2 = 124$;

Actual Exponent = $124 - 127 = -3$

Mantissa = $(1.0100\ 00\dots000)_2 = 1.25$ (1. is implicit)

So, the value of the decimal = $-1.25 \times 2^{-3} = -0.15625$

Floating Point

FP Example

Write -58.25_{10} in Single Precision Floating Point (IEEE 754)

1. Convert decimal to binary:

$$58.25_{10} = 111010.01_2$$

2. Write in normalized scientific notation:

$$1.1101001 \times 2^5 \text{ (1. is implicit)}$$

3. Fill in fields:

Sign bit: 1 (negative)

8 exponent bits: $(127 + 5) = 132 = 10000100_2$

23 fraction bits: 110 1001 0000 0000 0000 0000

1	100 0010 0	110 1001 0000 0000 0000 0000
---	------------	------------------------------

In Hexadecimal this is represented as 0xC2690000

Floating Point

FP Example

Write -58.25_{10} in Double Precision Floating Point (IEEE 754)

1. Convert decimal to binary:

$$58.25_{10} = 111010.01_2$$

2. Write in normalized scientific notation:

$$1.1101001 \times 2^5 \text{ (1. is implicit)}$$

3. Fill in fields:

Sign bit: 1 (negative)

11 exponent bits: $(1023 + 5) = 1028 = 100\ 0000\ 0100_2$

52 fraction bits: 1101 0010 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

0000	1	100 0000 0100	1101 0010 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
------	---	---------------	---

In Hexadecimal this is represented as 0xC04D200000000000

Floating Point

Smallest and Largest Normalised FP value

Single Precision FP:

Exponents 00000000 and 11111111 are reserved

Smallest value

Bias Exponent: 00000001

$\Rightarrow$ Actual Exponent = $1 - 127 = -126$

Fraction: 000...00 $\Rightarrow$ significand = 1.0

$\pm 1.0 \times 2^{-126} \approx \pm 1.17549... \times 10^{-38}$

Largest value

Biased Exponent: 11111110

$\Rightarrow$ Actual Exponent = $254 - 127 = +127$

Fraction: 111...11 $\Rightarrow$ significand ≈ 2.0

$\pm 2.0 \times 2^{+127} = 2^{-128} \approx \pm 3.4028... \times 10^{+38}$

Floating Point

Special Cases

Number	Sign	Exponent	Fraction
0	X	00000000	000000000000000000000000
∞	0	11111111	000000000000000000000000
$-\infty$	1	11111111	000000000000000000000000
NaN	X	11111111	non-zero

*NaN is Not a Number

Ex: $\div$ by zero, $\sqrt{-ve}$ no.

Floating Point

Special Cases

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1–254	Anything	1–2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

*Source: Computer Organisation & Design by Patterson & Hennessy, Morgan Kaufmann

Floating Point

Rounding Modes

- ❖ **Overflow:** number too large to be represented
- ❖ **Underflow:** number too small to be represented
- ❖ **Rounding modes:**
 - Down
 - Up
 - Toward zero
 - To nearest
- ❖ **Example:** round 1.100101 (1.578125) to only 3 fraction bits
 - Down: 1.100
 - Up: 1.101
 - Toward zero: 1.100
 - To nearest: 1.101 (1.625 is closer to 1.578125 than 1.5 is)

Floating Point

Think about it

- ❖ What are the largest normalized double precision FP numbers?
 - Hint: double precision exponent is 11 bits and mantissa is 52 bits

- ❖ What is the relative precision in terms of decimal fractional digits that single precision and double precision offer?
 - Hint: mantissa bits

- ❖ An example to represent denormalized valid floating point number?
 - Hint: Biased Exponent = 0 & Mantissa = Nonzero

Type	Mantissa bits
Single Precision (float)	23 bits
Double Precision (double)	52 bits

Formula to find decimal precision:

$$\text{Decimal digits of precision} \approx \text{mantissa bits} \times \log_{10}(2)$$

$$\log_{10}(2) \approx 0.3010$$

Type	Bits	Calculation	Approx. Decimal Digits
Single precision	23	23×0.3010	$\approx$ 6.9 digits
Double precision	52	52×0.3010	$\approx$ 15.7 digits

- **Single precision: ~ 7 decimal digits**
- **Double precision: ~ 16 decimal digits**

For denormalized numbers:

$$\text{Value} = (-1)^{\text{sign}} \times 0.\text{mantissa bits} \times 2^{-126}$$

exponent = 1 - bias = -126 for single precision)

Field	Binary	Meaning
Sign	0	positive
Exponent	00000000	denormalized
Mantissa	000000000000000000000001	smallest non-zero fraction
Value	$\approx 1.4 \times 10^{-45}$	smallest positive denormalized float

0 00000000 000000000000000000000001

Floating Point

MCQ

Given the following binary number in 32-bit (single precision) IEEE-754 format:

00111111001101101000000000000000

The decimal value closest to this floating-point number is

- a. $2.3 \times 10^{\text{power } -1}$
- b. $3.3 \times 10^{\text{power } -1}$
- c. $4.3 \times 10^{\text{power } -1}$
- d. $5.3 \times 10^{\text{power } -1}$

Floating Point

MCQ

Given the following binary number in 32-bit (single precision) IEEE-754 format:

00111110011011010000000000000000

The decimal value closest to this floating-point number is

- a. $2.3 \times 10^{\text{power } -1}$
- b. $3.3 \times 10^{\text{power } -1}$
- c. $4.3 \times 10^{\text{power } -1}$
- d. $5.3 \times 10^{\text{power } -1}$

Ans:

Exponent bits = E = 01111100 = 124 (in decimal)

Mantissa bits = M = 110110100000000000000000

In IEEE-754 format, 32-bit (single precision)

$$(-1)^s \times 1.M \times 2^{E-127}$$

$$= (-1)^0 \times 1.1101101 \times 2^{124-127}$$

$$= 1.1101101 \times 2^{-3}$$

$$= (1 + 2^{-1} + 2^{-2} + 2^{-4} + 2^{-5} + 2^{-7})$$

$$= 0.231 = 2.31 \times 10^{-1}$$

Floating Point

MCQ

IEEE floating point representation, the hexadecimal number 0xC0000000 corresponds to

- a. -3.0
- b. -1.0
- c. -4.0
- d. -2.0

Floating Point

MCQ

IEEE floating point representation, the hexadecimal number 0xC0000000 corresponds to

- a. -3.0
- b. -1.0
- c. -4.0
- d. -2.0

Ans:

Exponent bits = E = 10000000 = 128 (in decimal)

Mantissa bits = M = 000000000000000000000000

In IEEE-754 format, 32-bit (single precision)

$$(-1)^s \times 1.M \times 2^{E-127}$$

$$= (-1)^1 \times 1.0 \times 2^{128-127}$$

$$= -1 \times 1.0 \times 2$$

$$= -2.0$$



THANK YOU

Team DDCO

Department of Computer Science & Engineering



DIGITAL DESIGN & COMPUTER ORGANISATION

Floating Point

Team DDCO

Department of Computer Science
& Engineering

DIGITAL DESIGN & COMPUTER ORGANISATION

Some Fundamental Concepts

TEAM DDCO

Department of Computer Science and Engineering

Fundamental Concepts

Course Outline(T2- section 7.1)

- ❖ Some fundamental Concepts
- ❖ Register transfers
- ❖ Performing Arithmetic or Logic Operation
- ❖ Fetching a word from Memory
- ❖ Storing a word in memory

Fundamental Concepts

Some Fundamental Concepts(steps in execution of instruction)

- ❖ Fetch the contents of the memory location pointed to by the PC.
The contents of this location are interpreted as an instruction to be executed. Hence, they are loaded into the IR. Symbolically, this can be written as
 - **$IR \leftarrow [PC]$**
- ❖ Assuming that the memory is byte addressable, increment the contents of the PC by 4, that is,
 - **$PC = [PC] + 4$**
- ❖ Carry out the instruction specified by the instruction in the IR.

Fundamental Concepts

Some Fundamental Concepts

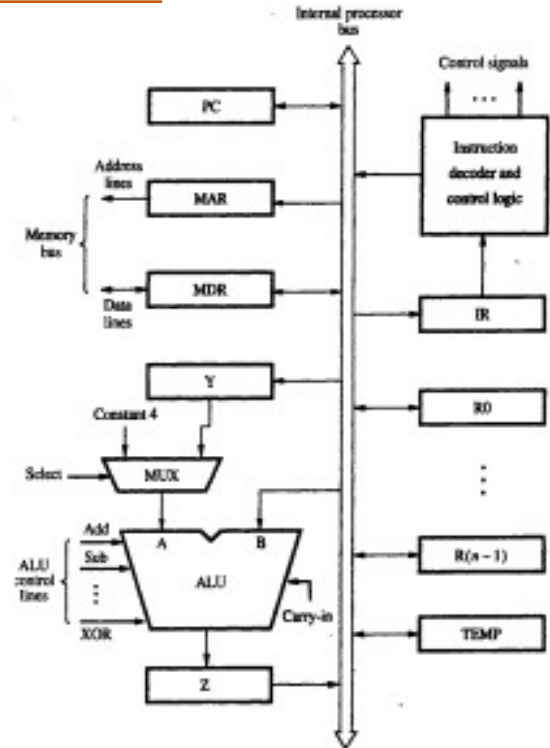
- ❖ **Instruction Execution Phases:** Instructions may occupy more than one word, requiring repeated fetch steps.
- ❖ **Fetch Phase:** Retrieves the complete instruction from memory.
- ❖ **Execution Phase:** Executes the instruction using the processor

Fundamental Concepts

Basic Processing Unit

Processor Components (Parts)

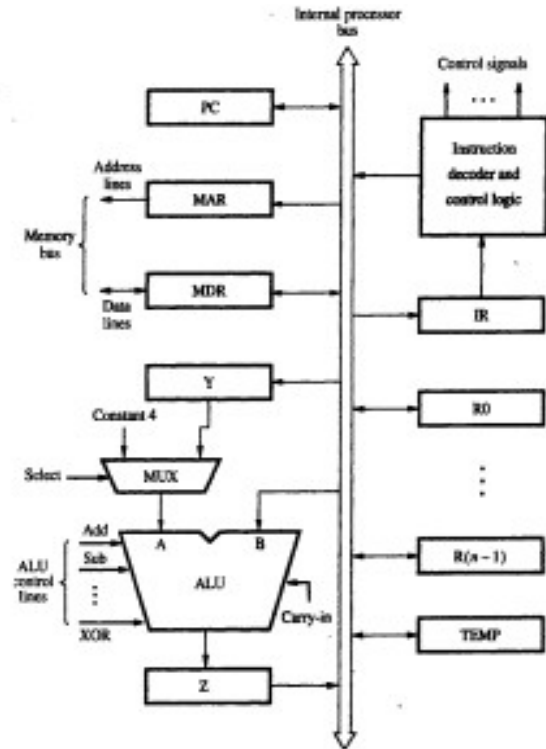
- ❖ Arithmetic and Logic Unit (ALU):
 - Responsible for performing arithmetic and logical operations within the processor.
- ❖ Processor Registers:
 - **General-purpose registers (R0 to Rn-1):** Used by programs for temporary data storage.
 - **Special-purpose registers:** Handle specific tasks, such as index registers and stack pointers.
 - **Temporary registers (Y, Z, TEMP):** Internal registers used during execution, not accessible to the programmer.



Fundamental Concepts

Basic Processing Unit

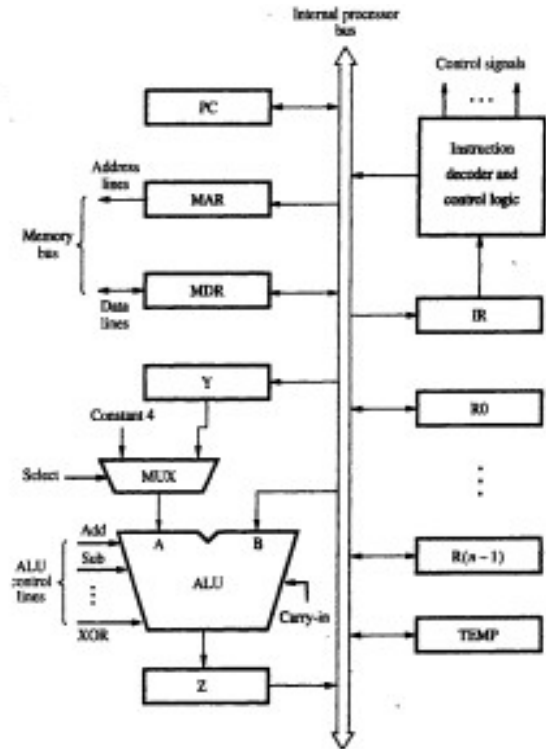
- ❖ Internal Bus:
 - A common **internal processor bus** connects all components like the ALU and registers for data transfer within the processor.
- ❖ Memory Data Register (MDR):
 - Interacts with both the memory bus and internal processor bus to transfer data between the processor and memory.
- ❖ Memory Address Register (MAR):
 - Holds the address of memory locations and interfaces between the internal processor bus and external memory bus.



Fundamental Concepts

Basic Processing Unit

- ❖ Multiplexer (MUX):
 - Selects between two inputs for the ALU:
 - **Register Y:** Provides input for arithmetic operations.
 - **Constant 4:** Used to increment the program counter (PC).
- ❖ Instruction Decoder and Control Logic:
 - Generates control signals for executing instructions.
 - Coordinates all internal units and manages memory bus interactions.
- ❖ Datapath:
 - **The collection of components like registers, ALU, and the internal bus, which together form the datapath of the processor.**



Fundamental Concepts

Basic Processing Unit

Instruction Execution Process

❖ Fetch Phase:

- The processor fetches the **instruction from memory**, possibly across multiple cycles if the instruction spans more than one word.
- The instruction is loaded into the **Instruction Register (IR)**.

❖ Data Transfer:

- Data can be transferred from one register to another or to the ALU for further operations.
- The **MUX** selects whether to provide input from **register Y** or the **constant 4** for the ALU, depending on the operation.

Fundamental Concepts

Basic Processing Unit

❖ **ALU Operations:**

- The ALU performs arithmetic or logical operations on the data from the selected input.
- The result of the operation is stored back in one of the processor's registers.

❖ **Memory Access:**

- To fetch data from memory, the **MAR** holds the address, and the **MDR** loads the data into a processor register.
- To store data in memory, the **MDR** sends data from a register to a specified memory location.

Fundamental Concepts

Basic Processing Unit

❖ **Instruction Execution:**

- The Instruction Decoder generates control signals to:
 - Select registers for data transfer.
 - Perform arithmetic or logic operations.
 - Fetch or store data from/to memory.
- The operation proceeds as a series of steps dictated by the instruction and controlled by the control logic unit.

❖ **Completion:**

- Once the instruction is executed, the program counter (PC) is incremented (by the constant value 4, selected by the MUX) to point to the next instruction, and the process repeats.

Fundamental Concepts

Register Transfer

Instruction Execution: Data transfers between registers.

Control Signals: Each register has two:

1. Ri_{in} : Loads data from the bus into the register.
2. Ri_{out} (**Riout**): Places the contents of the register onto the bus.

Example: Transfer R1 to R4:

3. Set $R1_{out} = 1$: Place data from R1 onto the bus.
4. Set $R4_{in} = 1$: Load bus data into R4.

Fundamental Concepts

Register Transfer

- ❖ **Clock Timing:** Transfers occur based on clock cycles. **Control signals are set** at the start of the clock cycle, and **registers load data** at the **next** active clock edge.
- ❖ **Register Structure:** Registers use **edge-triggered flip-flops**.
 - ❖ Data is loaded into flip-flops on the clock's rising edge.
 - ❖ Control signals return to 0 after the transfer.
- ❖ **Multiplexer & Tri-state Gate:**
 - ❖ **Multiplexer:** Selects either bus data or current register value for the flip-flop.
 - ❖ **Tri-state Gate:** Controls output from register to bus. When disabled, the bus is in a high-impedance (open-circuit) state.

Fundamental Concepts

Register Transfer

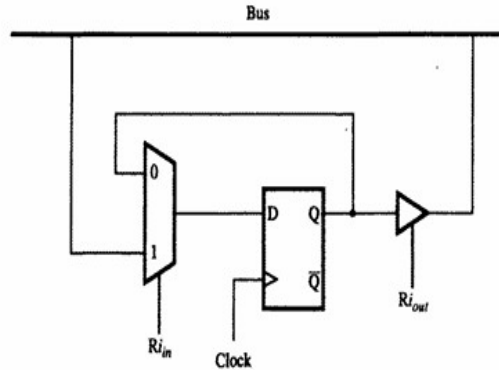


Figure 7.3 Input and output gating for one register bit.

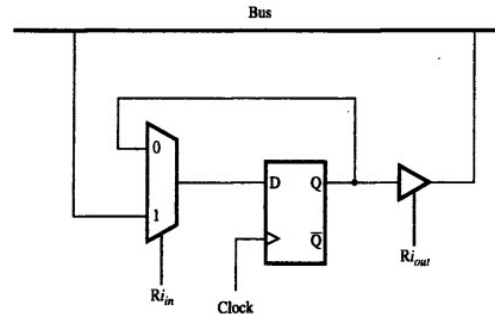
An implementation for one bit of register R_i is shown in Figure as an example. A two-input multiplexer is used to select the data applied to the input of an edge-triggered D flip-flop. When the control input R_{i_in} is equal to 1, the multiplexer selects the data on the bus. This data will be loaded into the flip-flop at the rising edge of the clock. When R_{i_in} is equal to 0, the multiplexer feeds back the value currently stored in the flip-flop.

The Q output of the flip-flop is connected to the bus via a tri-state gate. When R_{i_out} is equal to 0, the gate's output is in the high-impedance (electrically disconnected) state. This corresponds to the open-circuit state of a switch. When $R_{i_out} = 1$, the gate drives the bus to 0 or 1, depending on the value of Q.

Fundamental Concepts

Performing an Arithmetic or Logical Operation

- ❖ Each step corresponds to a clock cycle, with the respective control signals activated for that duration.
- ❖ **ALU Operations:** Controlled by specific signals (e.g., Add, Subtract). In reality, control signals are likely encoded for efficiency (e.g., 3 signals for 8 operations).



Fundamental Concepts

Performing an Arithmetic or Logical Operation

ALU: Performs arithmetic/logic operations; has no internal storage.

- Operands:
 - One from the **MUX**.
 - The other from the **bus**.
- Result stored in **register Z**.

Example: Adding R1 and R2, storing result in R3:

Step 1: Transfer **R1** to **Y**:

Activate **R1out**, **Yin**.

Step 2: Add **R1** (in **Y**) and **R2**:

Activate **R2out**, **SelectY**, **Add**, **Zin**.

Step 3: Transfer result from **Z** to **R3**:

Activate **Zout**, **R3in**.

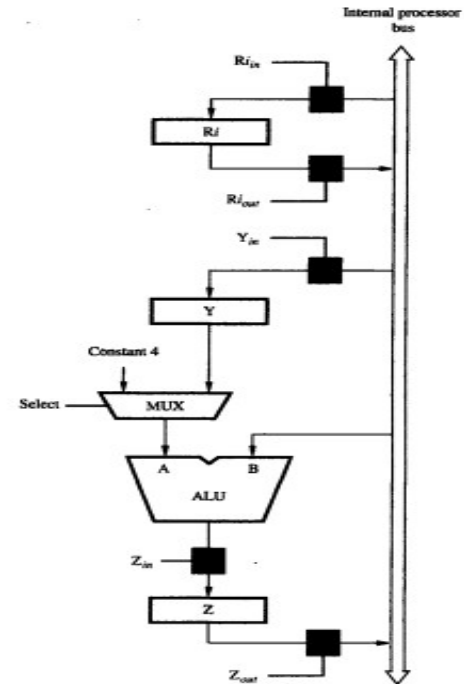


Figure 7.2 Input and output gating for the registers in Figure 7.1.

Fundamental Concepts

Fetching a Word from Memory (LOAD/READ)

Memory Fetch Process:

- ❖ The processor needs to **fetch data** (instruction or operand) from memory by specifying the **memory address**.
- ❖ The address is loaded into the **Memory Address Register (MAR)**, which is connected to the **memory bus**.
- ❖ A **Read** command is sent via control lines to initiate the operation.

Data Transfer via Memory Data Register (MDR):

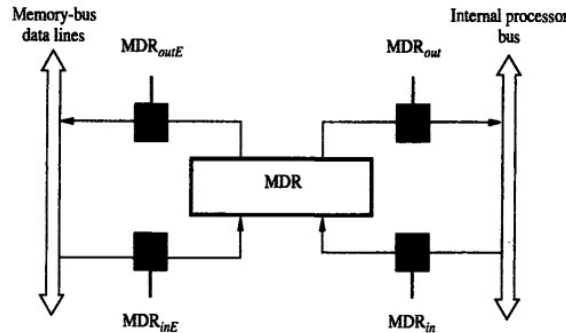
- ❖ The data fetched from memory is stored in the **Memory Data Register (MDR)**.
- ❖ Control signals:
 - **MDRin/MDRout** manage the connection to the internal bus.
 - **MDRinE/MDRoutE** control the connection to the external memory bus.

Fundamental Concepts

Fetching a Word from Memory

Waiting for Memory Response:

- ❖ The processor waits for a signal called **Memory-Function-Completed (MFC)**, which indicates that the requested data has been retrieved.
- ❖ While cache memory can respond in **1 clock cycle**, a miss or access to main memory or I/O registers may take **multiple cycles**.



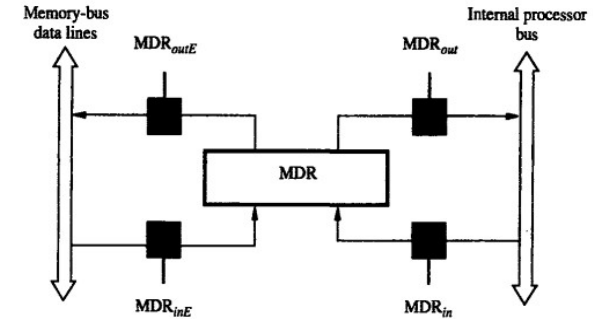
Fundamental Concepts

Fetching a Word from Memory

Example: Move (R1), R2 Instruction:

1. Transfer **R1** to **MAR** and initiate **Read**:
Control signals: R1out, MARin, Read.
2. Wait for **MFC** and load data from memory into **MDR**:
Control signals: MDRinE, wait for MFC.
3. Transfer MDR contents to R2:
Control signals: MDRout, R2in.

Combining Steps: Some actions, like loading the address and issuing the Read command, can be done in one step, reducing the total steps to **three** clock cycles.



1. $MAR \leftarrow [R1]$
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory bus
5. $R2 \leftarrow [MDR]$

Fundamental Concepts

Fetching a Word from Memory

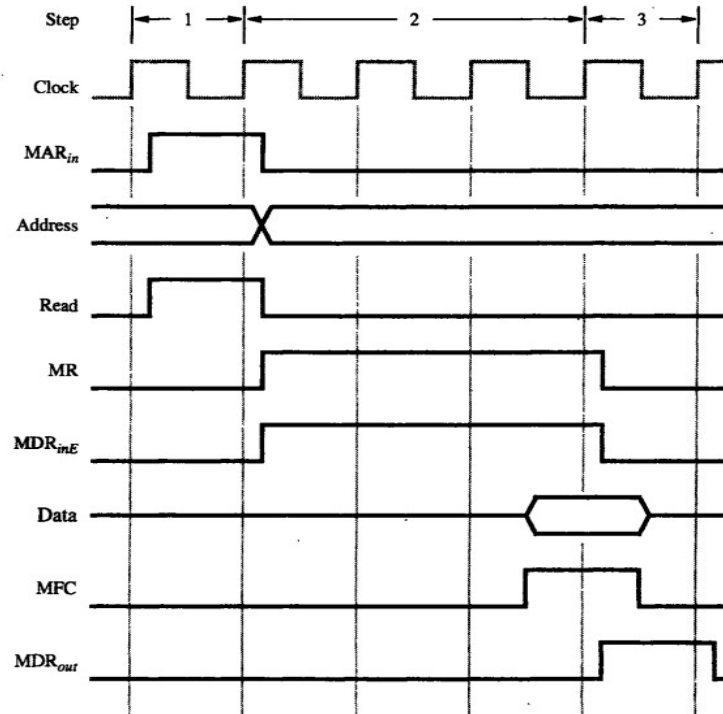


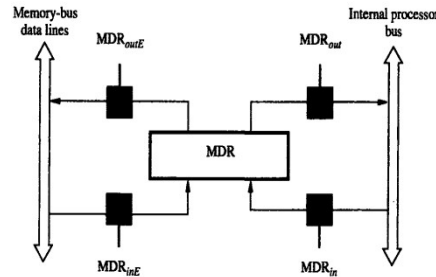
Figure 7.5 Timing of a memory Read operation.

Fundamental Concepts

Storing a word in memory

Writing a word into a memory location follows a similar procedure. The desired address is loaded into MAR. Then, the data to be written are loaded into MDR, and a Write command is issued. Hence, executing the instruction Move R2,(R1) requires the following sequence:

1. R1out, MARin
2. R2out, MDRin, Write
3. MDRoutE, WMFC



As in the case of the read operation, the Write control signal causes the memory bus interface hardware to issue a Write command on the memory bus. The processor remains in step 3 until the memory operation is completed and an MFC response is received.

Fundamental Concepts

MCQ

When the control signal $Ri_{in} = 1$ for a register, what operation is performed?

- A. The register places its contents on the bus.
- B. The bus data is loaded into the register.
- C. The register output enters high-impedance state.
- D. The register clears its contents.

During register transfer $R1 \rightarrow R2$, which control signal sequence is correct?

- A. $R1in=1$, $R2out=1$
- B. $R1out=1$, $R2in=1$
- C. $R1out=1$, $R2out=1$
- D. $R1in=1$, $R2in=1$

Fundamental Concepts

MCQ

When the control signal $R_{in} = 1$ for a register, what operation is performed?

- A. The register places its contents on the bus.
- B. The bus data is loaded into the register.
- C. The register output enters high-impedance state.
- D. The register clears its contents.

Answer: B

Explanation: $R_{in} = 1$ activates the input control, causing the D-flip-flops of the register to load bus data on the next clock edge.

During register transfer $R1 \rightarrow R2$, which control signal sequence is correct?

- A. $R1_{in}=1$, $R2_{out}=1$
- B. $R1_{out}=1$, $R2_{in}=1$
- C. $R1_{out}=1$, $R2_{out}=1$
- D. $R1_{in}=1$, $R2_{in}=1$

Answer: B

Explanation: $R1_{out}$ enables the source register onto the bus; $R2_{in}$ loads bus data into destination register

Fundamental Concepts

MCQ

During a memory read sequence using MAR and MDR, which control signals are correctly asserted first?

- A. MARin then Read
- B. Read then MARin
- C. MDRin then MARin
- D. MDRout then MARin

What is the role of a tri-state gate in register organization?

- A. Amplifies the bus signal.
- B. Acts as a decoder for address selection.
- C. Connects or isolates register output from the common bus.
- D. Generates clock pulses for register transfer.

Fundamental Concepts

MCQ

During a memory read sequence using MAR and MDR, which control signals are correctly asserted first?

- A. MARin then Read
- B. Read then MARin
- C. MDRin then MARin
- D. MDRout then MARin

Answer: A

Explanation: The address must be loaded into MAR before issuing the Read command.

What is the role of a tri-state gate in register organization?

- A. Amplifies the bus signal.
- B. Acts as a decoder for address selection.
- C. Connects or isolates register output from the common bus.
- D. Generates clock pulses for register transfer.

Answer: C

Explanation: The tri-state buffer enables or disconnects the register from the bus without interfering with other devices.



THANK YOU

Team DDCO

Department of Computer Science & Engineering



DIGITAL DESIGN & COMPUTER ORGANISATION

Execution of Complete Instruction

Team DDCO

Department of Computer Science
& Engineering

Execution of Complete Instruction, Single Cycle Datapath

Course Outline

- ❖ Execution of a complete instruction, Multiple Bus Organization

Execution of Complete Instruction, Single Cycle Datapath

Execution of Complete Instruction-section 7.2, T2

Add (R3), R1

- ❖ Fetch the instruction
- ❖ Fetch the first operand (the contents of the memory location pointed to by R3)
- ❖ Perform the addition
- ❖ Load the result into R1

Execution of Complete Instruction

Execution of Complete Instruction

Step	Action
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMFC
3	MDR_{out} , IR_{in}
4	$R3_{out}$, MAR_{in} , Read
5	$R1_{out}$, Y_{in} , WMFC
6	MDR_{out} , SelectY, Add, Z_{in}
7	Z_{out} , $R1_{in}$, End

Figure 7.6. Control sequence for execution of the instruction Add (R3),R1.

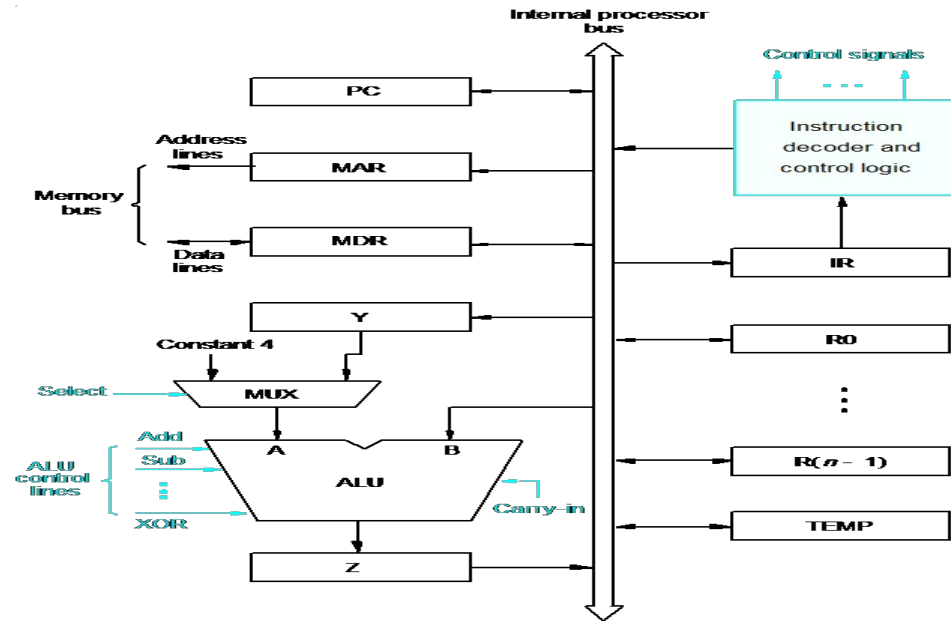


Figure 7.1. Single-bus organization of the datapath inside a processor

Execution of Complete Instruction

Execution of Branch Instructions- (section 7.2.1 T2)

- ❖ A branch instruction replaces the contents of PC with the branch target address, which is usually obtained by adding an offset X given in the branch instruction.
- ❖ The offset X is usually the difference between the branch target address and the address immediately following the branch instruction.

Execution of Complete Instruction

Execution of Branch Instructions

Control sequence for an unconditional branch instruction

Step Action

- 1 PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
- 2 Z_{out} , PC_{in} , Y_{in} , WMFC
- 3 MDR_{out} , IR_{in}
- 4 Offset-field-of- IR_{out} , Add, Z_{in}
- 5 Z_{out} , PC_{in} , End

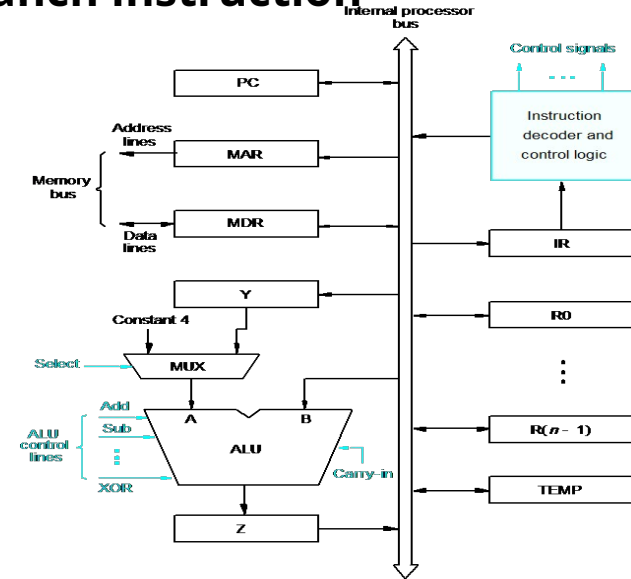


Figure 7.1. Single-bus organization of the datapath inside a processor

Execution of Complete Instruction

Execution of Branch Instructions

- For example, if the branch instruction is at location 2000 and if the branch target address is 2050, the value of X must be 46.
- Consider now a conditional branch. In this case, we need to check the status of the condition codes before loading a new value into the PC. For example, for a Branch-on-negative (Branch < 0) instruction, step 4 is replaced with
- **Offset-field-of-IRout, Add, Zin, If $N = 0$ then End**
- Thus, if $N = 0$ the processor returns to step 1 immediately after step 4. If $N = 1$, step 5 is performed to load a new value into the PC, thus performing the branch operation.

Execution of Complete Instruction

Multiple Bus Organization-Section 7.3 ,T2

- ❖ **It is a system architecture used in computer systems to enhance data transfer and processing efficiency by utilizing multiple buses.**
 - Allow the contents of two different registers to be accessed simultaneously and have their contents placed on buses A and B
 - Allow the data on bus C to be loaded into a third register during the same clock cycle.

Execution of Complete Instruction

Multiple Bus Organization

- ❖ General purpose registers are combined into a single block called **register file**.
- ❖ 3 ports, 2 output ports – access two different registers and have their contents on buses A and B
- ❖ Third port allows data on bus c during same clock cycle.

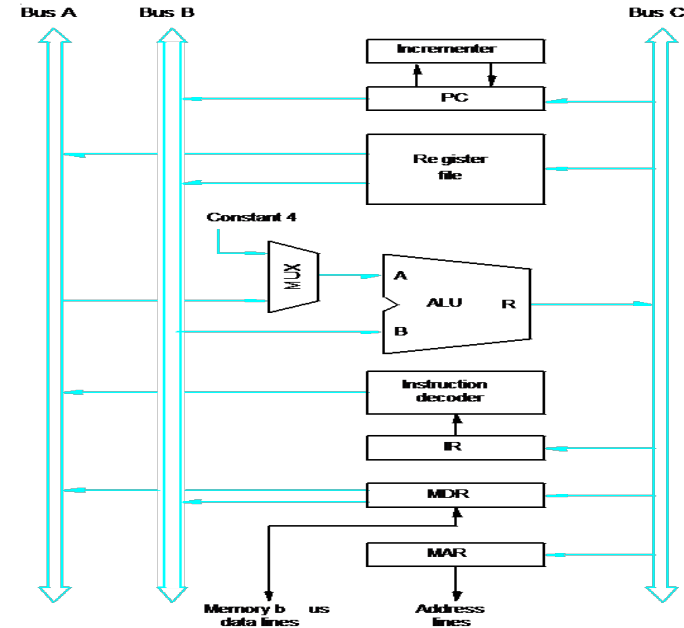


Figure 7.8. Three-bus organization of the datapath.

Execution of Complete Instruction

Multiple Bus Organization

- ❖ The Arithmetic Logic Unit (ALU) can output one of its two input operands directly to bus C without modification. **This operation is controlled by ALU signals such as $R = A$ or $R = B$.**
- ❖ A dedicated incrementer unit is used to advance the **Program Counter (PC) by 4**, bypassing the need for the main ALU to perform this operation by adding the constant value of 4.
- ❖ The inclusion of the incrementer reduces the load on the main ALU, streamlining the process.

Execution of Complete Instruction

Multiple Bus Organization

Add R4, R5, R6

Step Action

- 1 PC_{out}, R=B, MAR_{in}, Read, IncPC
- 2 WMFC
- 3 MDR_{out}B, R=B, IR_{in}
- 4 R4_{out}A, R5_{out}B, SelectA, Add, R6_{in}, End

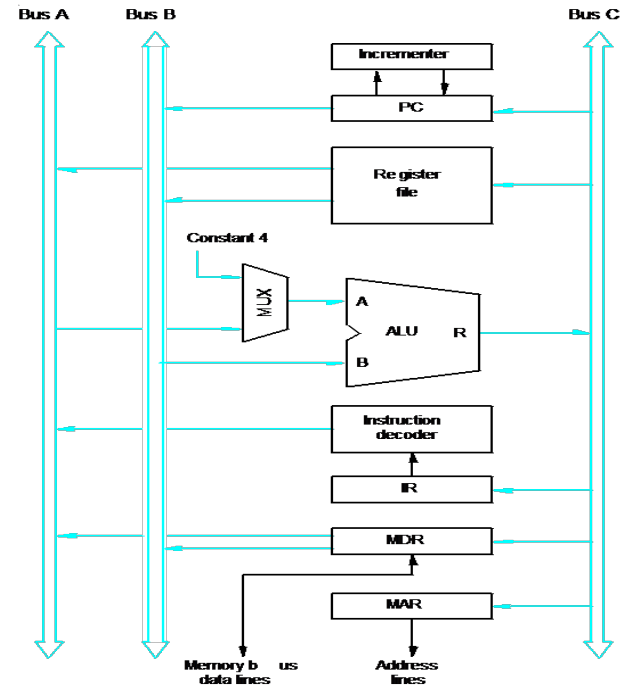


Figure 7.8. Three-bus organization of the datapath.

Execution of Complete Instruction

Multiple Bus Organization

- The introduction of the Incrementer unit, which is used to increment the PC by 4.
- Using the Incrementer eliminates the need to add 4 to the PC using the main ALU, as was done earlier.
- The source for the constant 4 at the ALU input multiplexer is still useful. It can be used to increment other addresses, such as the memory addresses in LoadMultiple and StoreMultiple instructions
- In step 1, the contents of the PC are passed through the ALU, using the R=B control signal, and loaded into the MAR to start a memory read operation. At the same time the PC is incremented by 4.
- Note that the value loaded into MAR is the original contents of the PC. The incremented value is loaded into the PC at the end of the clock cycle and will not affect the contents of MAR.
- In step 2, the processor waits for MFC and loads the data received into MDR, then transfers them to IR in step 3. Finally, the execution phase of the instruction requires only one control step to complete, step 4.

Execution of Complete Instruction

MCQ

In a branch instruction, the Program Counter (PC) is replaced by the _____.

- A. Instruction Register (IR) contents
- B. Branch Target Address
- C. Condition Code Register value
- D. Offset value X only

Which of the following control sequence corresponds to a conditional branch operation?

- A. Offset-field-of-IRout, Add, Zin, If N = 0 then End
- B. Rout, MARin, Read, WMFC
- C. Yin, Add, Zout, Rin
- D. PCout, MARin, Read

Execution of Complete Instruction

MCQ

In a branch instruction, the Program Counter (PC) is replaced by the _____.

- A. Instruction Register (IR) contents
- B. Branch Target Address
- C. Condition Code Register value
- D. Offset value X only

Answer: B

Explanation: A branch instruction replaces the PC contents with the branch target address, which is obtained by adding the offset X to the next sequential address.

Which of the following control sequence corresponds to a conditional branch operation?

- A. Offset-field-of-IRout, Add, Zin, If N = 0 then End
- B. Rout, MARin, Read, WMFC
- C. Yin, Add, Zout, Rin
- D. PCout, MARin, Read

Answer: A

Explanation: This sequence computes the branch target and loads it into Z only if the condition code is true.

Execution of Complete Instruction

MCQ

Which of the following register sequences is correct for fetching an instruction?

- A. PCout → MARin, Read, WMFC, MDRout → IRin
- B. IRout → PCin, MDRout → MARin
- C. MDRin → PCout → IRin
- D. Zout → Yin → MARin

What is the role of the Incrementer unit in the processor datapath?

- A. To add two register values
- B. To add 4 to the PC without using the main ALU
- C. To store ALU results temporarily
- D. To control the tri-state bus signals

Execution of Complete Instruction

MCQ

Which of the following register sequences is correct for fetching an instruction?

- A. PCout → MARin, Read, WMFC, MDRout → IRin
- B. IRout → PCin, MDRout → MARin
- C. MDRin → PCout → IRin
- D. Zout → Yin → MARin

Answer: A

Explanation: This represents the typical fetch sequence used in single-cycle datapaths.

What is the role of the Incrementer unit in the processor datapath?

- A. To add two register values
- B. To add 4 to the PC without using the main ALU
- C. To store ALU results temporarily
- D. To control the tri-state bus signals

Answer: B *Explanation:* The Incrementer bypasses the main ALU for PC increment operations, reducing load and latency



THANK YOU

Team DDCO

Department of Computer Science & Engineering



DIGITAL DESIGN & COMPUTER ORGANISATION

Hardwired Control, Micro programmed control

Team DDCO

Department of Computer Science
& Engineering

DIGITAL DESIGN & COMPUTER ORGANISATION

Hardwired Control, Microprogrammed Control

Team DDCO.

Department of Computer Science & Engineering

Hardwired Control, Microprogrammed Control

Course Outline

- ❖ Hardwired Control
- ❖ A Complete Processor
- ❖ Microprogrammed Control

Hardwired Control, Microprogrammed Control

Hardwired Control(section 7.4 ,T2)

To execute instructions, the processor must have some **means of generating the control signals** needed in the proper sequence.

Computer designers use a wide variety of techniques to solve this problem. The approaches used fall into one of two categories:

- ❖ hardwired control
- ❖ microprogrammed control

Hardwired Control, Microprogrammed Control

Hardwired Control

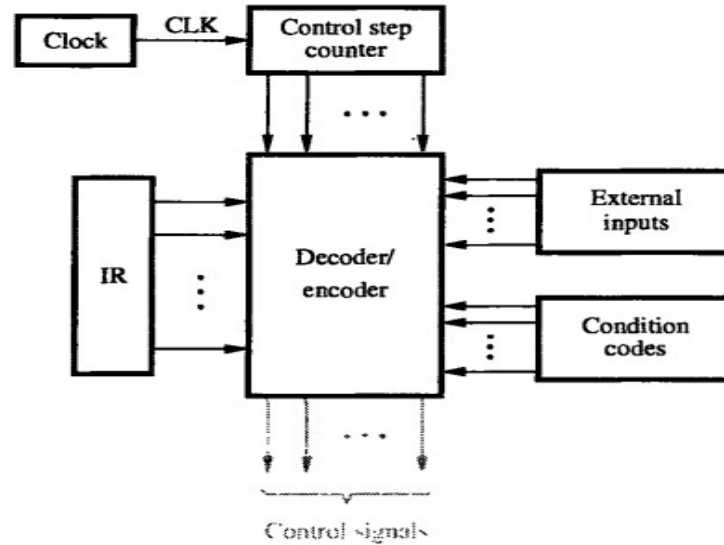


Figure 7.10 Control unit organization.

Hardwired Control, Microprogrammed Control

Hardwired Control

Hardwired control directly wires a fixed sequence of control signals for each instruction. These signals are completed in one clock cycle per step.

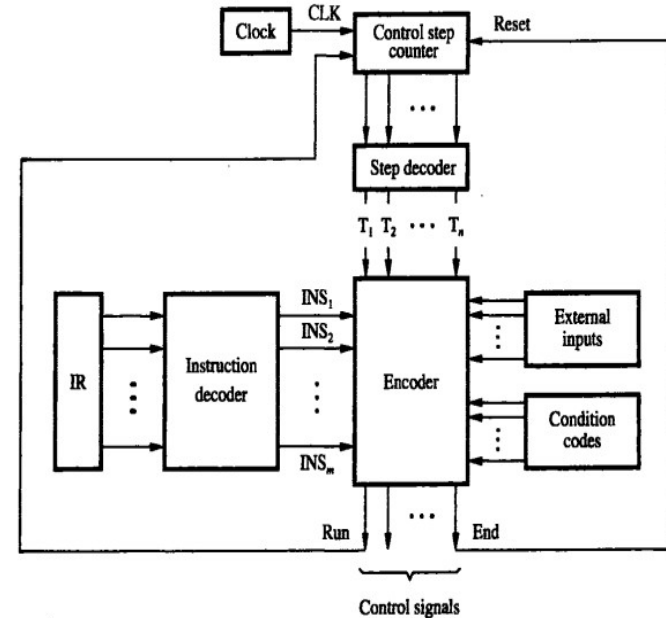
A **control step counter** keeps track of the control steps. Control signals are determined based on:

1. The control step counter.
2. The instruction register.
3. Condition code flags.
4. External inputs like **Memory Function Completed (MFC)** and **interrupts**.

Hardwired Control, Microprogrammed Control

Hardwired Control

- ❖ The End signal resets the control step counter to begin a new instruction cycle. Another signal, RUN, increments the counter each clock cycle, but halts if RUN is set to 0 (useful when waiting for memory responses).
- ❖ The control unit's structure consists of a decoder/encoder block that generates control outputs.
 - A step decoder provides a unique signal for each control step.
 - An instruction decoder activates the signal corresponding to a specific instruction.
 - RUN causes the counter to be incremented by one at the end of every clock cycle. When RUN is equal to 0, the counter stops counting. This is needed whenever the WMFC signal is issued, to cause the processor to wait for the reply from the memory.



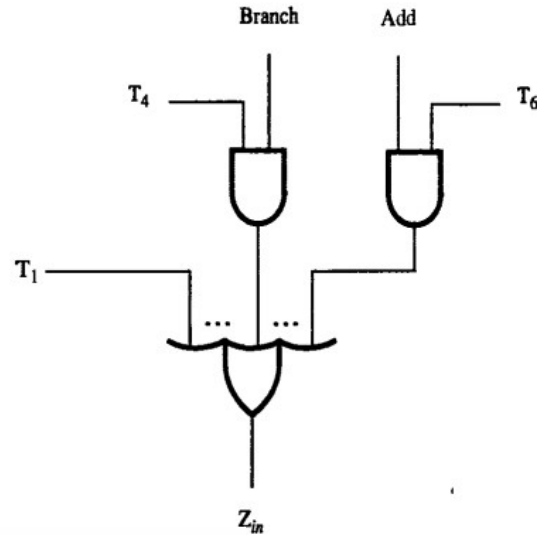
Hardwired Control, Microprogrammed Control

Hardwired Control

- ❖ The **encoder** combines these signals to produce control outputs like **Yin**, **PCout**, **Add**, and **End**.
- ❖ For example, the **Zin** control signal is activated depending on the instruction and control step (e.g., T1 for all instructions, T6 for Add). This logic ensures correct actions are taken during execution.
- ❖ The control unit operates as a **state machine**, transitioning between states each clock cycle, based on the instruction register, condition codes, and external inputs.
- ❖ **Hardwired control** offers fast operation but has limited flexibility and can only handle a restricted set of complex instructions.

Hardwired Control, Microprogrammed Control

Hardwired Control



$$Z_{in} = T_1 + T_6 \cdot ADD + T_4 \cdot BR + \dots$$

Hardwired Control, Microprogrammed Control

Hardwired Control

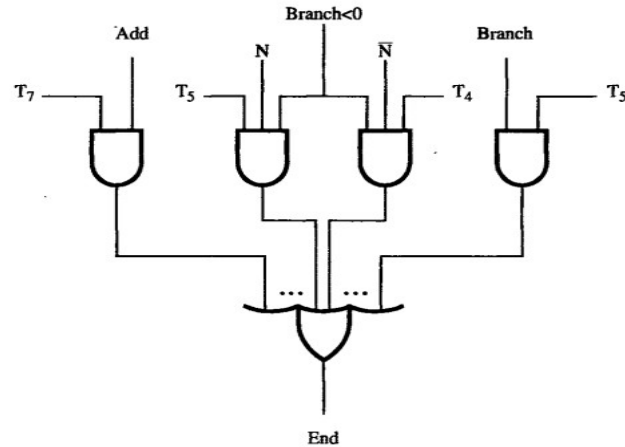


Figure 7.13 Generation of the End control signal.

$$\text{End} = T_7 \cdot \text{ADD} + T_5 \cdot \text{BR} + (T_5 \cdot N + T_4 \cdot \bar{N}) \cdot \text{BRN} + \dots$$

Hardwired Control, Microprogrammed Control

A Complete Processor(section 7.4.1,T2)

- ❖ **Processor Structure:** The processor consists of various units including an **instruction unit, integer and floating-point processing units, and memory caches.**
- ❖ **Instruction Unit:** Fetches instructions from either the **instruction cache** or **main memory** (if the instruction isn't in the cache).
- ❖ **Separate Processing Units:** The processor has **distinct units** for handling **integer data** and **floating-point data**, each organized similarly to earlier discussed structures.
- ❖ **Data Cache:** A **data cache** is placed between the processing units and main memory, ensuring faster access to data during operations.
- ❖ **Cache Configuration:** Modern processors often use **separate caches** for instructions and data. Some processors combine them into a **single unified cache.**

Hardwired Control, Microprogrammed Control

A Complete Processor

- ❖ **Bus Interface:** The processor is connected to the rest of the system through a **bus interface**, which links it to the system bus and other components.
- ❖ **Multiple Processing Units:** To boost performance, processors may include **multiple integer and floating-point units** to allow for parallel or concurrent processing.

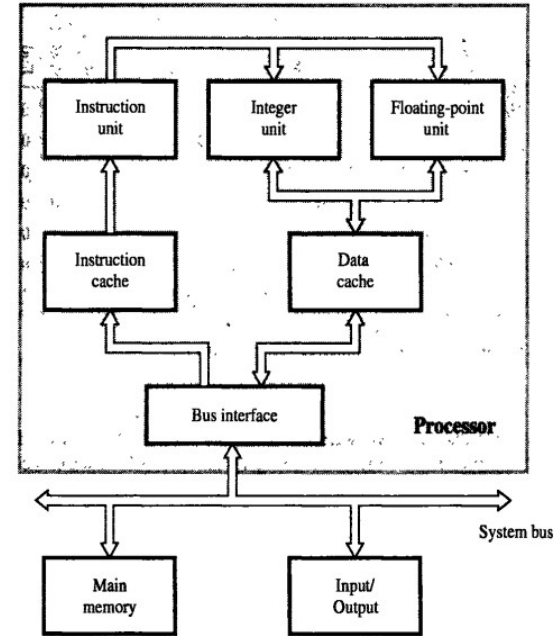


Figure 7.14 Block diagram of a complete processor.

Hardwired Control, Microprogrammed Control

Microprogrammed Control-section 7.5,T2

- ❖ **Microprogrammed Control Concept:** Unlike hardwired control, microprogrammed control generates control signals using a **program-like approach**. Control signals are stored as **control words (CWs)**, which resemble machine language programs.
- ❖ **Control Word (CW):** A **control word** is a binary word where each bit represents a specific control signal within the processor. Each control step in an instruction's sequence is represented by a unique CW.

Hardwired Control, Microprogrammed Control

Microprogrammed Control

Micro - instruction	..	PC _{in}	PC _{out}	MAR _{in}	Read	MDR _{out}	IR _{in}	Y _{in}	Select	Add	Z _{in}	Z _{out}	R1 _{out}	R1 _{in}	R3 _{out}	WMFC	End	:
1		0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	
2		1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	
3		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	
4		0	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	
5		0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0	
6		0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	
7		0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	

Figure 7.15 An example of microinstructions for Figure 7.6.

Select y is represented as select=0
Select 4 is represented as select=1

Step Action

- 1 PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
- 2 Z_{out}, PC_{in}, Y_{in}, WMFC
- 3 MDR_{out}, IR_{in}
- 4 R3_{out}, MAR_{in}, Read
- 5 R1_{out}, Y_{in}, WMFC
- 6 MDR_{out}, SelectY, Add, Z_{in}
- 7 Z_{out}, R1_{in}, End

Figure 7.6 Control sequence for execution of the instruction Add (R3),R1.

Hardwired Control, Microprogrammed Control

Microprogrammed Control

- ❖ **Conditional Branch Microinstructions:** Microprogrammed control can handle decision-making using **conditional branch microinstructions**. These microinstructions allow the control unit to check condition codes, external inputs, or instruction bits and branch to a different microinstruction sequence based on these conditions.
- ❖ **Branch Instruction Example:** The instruction **Branch<0** can be implemented through a microroutine that checks the **N (negative)** condition code bit. **If the bit is 0, control branches to fetch a new instruction. Otherwise, the target branch address is loaded into the PC.**
- ❖ **Advantages:** Microprogrammed control provides flexibility and makes it easier to modify and implement complex instruction sets. However, it may be slower than hardwired control due to the sequential reading of microinstructions.

Hardwired Control, Microprogrammed Control

Microprogrammed Control

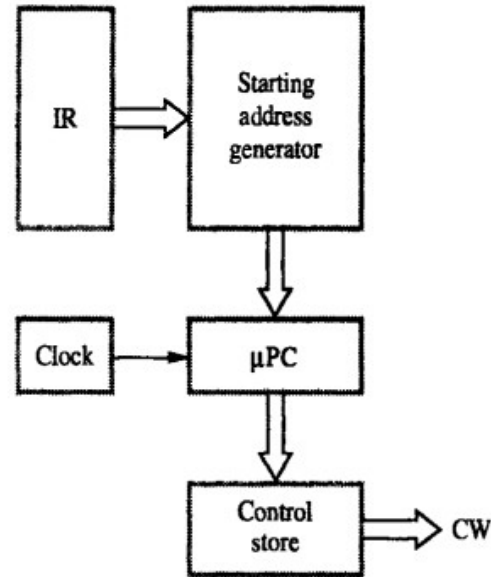


Figure 7.16 Basic organization of a microprogrammed control unit.

Hardwired Control, Microprogrammed Control

Microprogrammed Control

Address	Microinstruction
0	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
1	Z_{out} , PC_{in} , Y_{in} , WMFC
2	MDR_{out} , IR_{in}
3	Branch to starting address of appropriate microroutine
.....	
25	If $N=0$, then branch to microinstruction 0
26	Offset-field-of- IR_{out} , SelectY, Add, Z_{in}
27	Z_{out} , PC_{in} , End

Figure 7.17 Microroutine for the instruction Branch < 0.

Hardwired Control, Microprogrammed Control

Microprogrammed Control

To support microprogram branching, the control unit is modified as shown in Figure 7.18. (next slide). The starting address generator from Figure 7.16 becomes the starting and branch address generator. This block loads a new address into the uPC when instructed by a microinstruction. Inputs to this block include external inputs, condition codes, and the contents of the instruction register for conditional branching.

The uPC increments with each new microinstruction, except in these cases:

1. When a new instruction is loaded into the IR, the uPC loads the micro routine's starting address.
2. When a Branch micro instruction's condition is satisfied, the uPC loads the branch address.
3. For an End microinstruction, the uPC loads the address of the first control word (CW) in the instruction fetch cycle (address 0 in Figure 7.17).

Hardwired Control, Microprogrammed Control

Microprogrammed Control

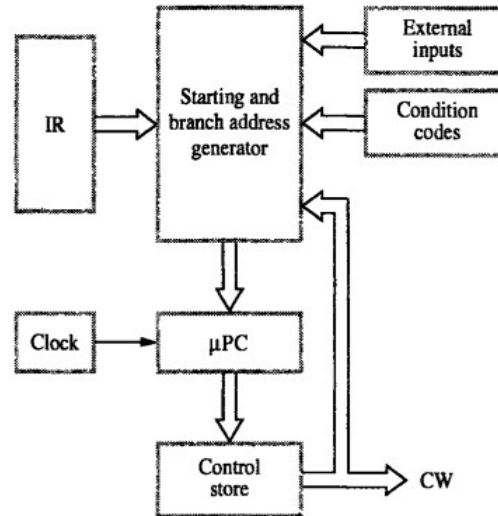


Figure 7.18 Organization of the control unit to allow conditional branching in the microprogram.

Hardwired Control, Microprogrammed Control

Microinstructions

Microinstruction Format:

A basic scheme assigns one bit for each control signal, **but this leads to long microinstructions** due to a large number of signals. In a processor, only a few control signals are active at a time, **leading to inefficient bit usage**.

Control Signal Grouping:

Control signals can be grouped based on **mutual exclusivity** (e.g., ALU operations, register outputs). A binary code represents the signals in each group, reducing the number of bits needed per microinstruction.

Hardwired Control, Microprogrammed Control

Microinstructions(20 bit for 42 signals)(Horizontal encoding)

Microinstruction

F1	F2	F3	F4	F5
F1 (4 bits)	F2 (3 bits)	F3 (3 bits)	F4 (4 bits)	F5 (2 bits)
0000: No transfer	000: No transfer	000: No transfer	0000: Add	00: No action
0001: PC _{out}	001: PC _{in}	001: MAR _{in}	0001: Sub	01: Read
0010: MDR _{out}	010: IR _{in}	010: MDR _{in}	⋮	10: Write
0011: Z _{out}	011: Z _{in}	011: TEMP _{in}	⋮	
0100: R0 _{out}	100: R0 _{in}	100: Y _{in}	1111: XOR	
0101: R1 _{out}	101: R1 _{in}		⏟	
0110: R2 _{out}	110: R2 _{in}		16 ALU functions	
0111: R3 _{out}	111: R3 _{in}			
1010: TEMP _{out}				
1011: Offset _{out}				

Hardwired Control, Microprogrammed Control

Microinstructions(20 bit for 42 signals)(Horizontal encoding)

F6	F7	F8	...
F6 (1 bit)	F7 (1 bit)	F8 (1 bit)	
0: SelectY 1: Select4	0: No action 1: WMFC	0: Continue 1: End	

Figure 7.19 An example of a partial format for field-encoded microinstructions.

Hardwired Control, Microprogrammed Control

Microinstructions

Horizontal vs. Vertical Organization:

- **Horizontal Organization: Minimally encoded;** more control signals can be activated in parallel (**faster** execution but longer microinstructions).
- **Vertical Organization: Highly encoded;** fewer bits in each microinstruction, but **slower** due to sequential execution of control functions.

Hardware Trade-off:

More encoding (vertical) reduces microinstruction length but increases the complexity of decoding hardware. Less encoding (horizontal) allows faster execution but requires more bits in each microinstruction.

Hardwired Control, Microprogrammed Control

MCQ

In a hardwired control unit, the control signals for each instruction are generated based on:

- A. Microprograms stored in ROM
- B. Control step counter, instruction register, and condition flags
- C. Address decoding logic in the MAR
- D. A software loop maintained in main memory

Which of the following statements about hardwired control is correct?

- A. It is flexible and easy to modify for new instructions.
- B. It executes microinstructions sequentially.
- C. It is faster but less flexible than microprogrammed control
- D. It stores microinstructions in control memory.

Hardwired Control, Microprogrammed Control

MCQ

In a hardwired control unit, the control signals for each instruction are generated based on:

- A. Microprograms stored in ROM
- B. Control step counter, instruction register, and condition flags
- C. Address decoding logic in the MAR
- D. A software loop maintained in main memory

Answer: B

Explanation: Hardwired control uses combinational logic driven by the control-step counter, instruction decoder, and condition codes

Which of the following statements about hardwired control is correct?

- A. It is flexible and easy to modify for new instructions.
- B. It executes microinstructions sequentially.
- C. It is faster but less flexible than microprogrammed control
- D. It stores microinstructions in control memory.

Answer: C, Explanation: Hardwired control achieves high speed through fixed logic but is difficult to modify for complex or evolving instruction sets

Hardwired Control, Microprogrammed Control

MCQ

A **Control Word (CW)** in a microprogrammed unit:

- A. Is an instruction for the ALU
- B. Represents a set of simultaneous control signals
- C. Contains only address bits of data memory
- D. Holds flag register values

The **microprogram counter (uPC)** in a microprogrammed control unit increments with each microinstruction except when:

- A. $MFC = 0$
- B. A Branch μ -instruction or End μ -instruction occurs
- C. $RUN = 1$
- D. $WMFC = 1$

Hardwired Control, Microprogrammed Control

MCQ

A **Control Word (CW)** in a microprogrammed unit:

- A. Is an instruction for the ALU
- B. Represents a set of simultaneous control signals
- C. Contains only address bits of data memory
- D. Holds flag register values

Answer: B

Explanation: Each CW specifies which control lines (e.g., PCout, Yin, Add) are active during a micro-operation

The **microprogram counter (uPC)** in a microprogrammed control unit increments with each microinstruction except when:

- A. $MFC = 0$
- B. A Branch μ -instruction or End μ -instruction occurs
- C. $RUN = 1$
- D. $WMFC = 1$

Answer: B

Explanation: For branch or end microinstructions, the uPC loads a new or starting address instead of incrementing sequentially



THANK YOU

Team DDCO

Department of Computer Science & Engineering